



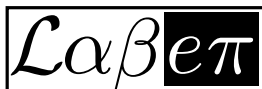
Universidade Federal do Rio Grande do Norte – UFRN
Centro de Ensino Superior do Seridó – CERES
Departamento de Computação e Tecnologia – DCT
Bacharelado em Sistemas de Informação – BSI

Boilerplate para Ferramentas de Visualização e Análise de Dados

Zaú Júlio Araújo Galvão

Orientador: Prof. Me. Luiz Paulo de Assis Barbosa

Trabalho de Conclusão de Curso apresentado ao Curso de Bacharelado em Sistemas de Informação como parte dos requisitos para obtenção do título de Bacharel em Sistemas de Informação.



Laboratório de Elementos do Processamento da Informação – LabEPI
Caicó, RN, 31 de janeiro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Galvão, Zaú Júlio Araújo.

Boilerplate para ferramentas de visualização e análise de dados / Zaú Júlio Araújo Galvão. - 2025.
68f.: il.

Trabalho de Conclusão de Curso - TCC (graduação) -
Universidade Federal do Rio Grande do Norte, Centro de Ensino
Superior do Seridó, Bacharelado em Sistemas de Informação,
Natal, 2025.

Orientação: Prof. Me. Luiz Paulo de Assis Barbosa.

1. Boilerplate - TCC. 2. Análise de Dados - TCC. 3.
Visualização de Dados - TCC. 4. Aprendizado de Máquina - TCC. 5.
Tomada de Decisão - TCC. I. Barbosa, Luiz Paulo de Assis. II.
Título.

RN/UF/BCZM

CDU 004.4

Elaborado por Jackeline dos Santos Pinheiro da Silva Maia
Cavalcanti - CRB-15/317

11
12

Boilerplate para Ferramentas de Visualização e Análise de Dados

13

Zaú Júlio Araújo Galvão

14
15

Monografia aprovada em 31 de janeiro de 2025 pela banca examinadora composta pelos
seguintes membros:

16
17

Prof. Me. Luiz Paulo de Assis Barbosa (orientador) DCT/UFRN

18
19

Prof. Dr. João Paulo de Souza Medeiros DCT/UFRN

20
21

Prof. Dr. João Batista Borges Neto DCT/UFRN

“Logic is the beginning of wisdom, not the end of it.”
Spock, Star Trek VI: The Undiscovered Country

23 Agradecimentos

24 Aos Professores:

25 Agradeço ao professor Luiz Paulo pela oportunidade de ingressar na pesquisa científica
26 logo no início da faculdade e por sua paciência e orientação ao longo dos anos. Suas
27 contribuições foram essenciais para o meu desenvolvimento acadêmico.

28 Ao professor João Paulo, expresse minha gratidão pelas valiosas orientações e por ser
29 uma fonte constante de inspiração. Sua dedicação e visão sempre me motivaram a buscar
30 a excelência em minha jornada acadêmica e profissional.

31 Ao professor João Borges, agradeço pelas aulas de Redes e Sistemas Operacionais,
32 que foram verdadeiras referências educacionais. Suas abordagens detalhadas e a clareza
33 no ensino me ajudaram a compreender, aprofundar e despertar o interesse por esses te-
34 mas. Suas contribuições me forneceram uma perspectiva nova e valiosa sobre o mundo da
35 computação. Obrigado por compartilhar seu conhecimento e experiência comigo.

36 Aos Colegas:

37 Agradeço aos meus colegas pela parceria e companheirismo ao longo de toda a tra-
38 jetória acadêmica. A experiência adquirida na empresa júnior Byte Seridó Jr. foi enri-
39 quecedora, e sou grato pela oportunidade de trabalhar ao lado de pessoas talentosas e
40 dedicadas, dentro e fora da academia.

41 Ao Laboratório:

42 Finalmente, sou grato pela oportunidade de desenvolver este trabalho no Laboratório
43 de Elementos do Processamento da Informação (LabEPI), sediado no Centro de Ensino
44 Superior do Seridó da Universidade Federal do Rio Grande do Norte.

Resumo

46

47

48

49

50

51

52

53

54

55

56

No presente trabalho é descrito o desenvolvimento de um *boilerplate* voltado à criação de aplicações e ferramentas interativas para análise e visualização de dados. O software desenvolvido oferece uma infraestrutura modular e extensível, integrando `GTK` para interface gráfica, `matplotlib` para visualização, `pandas` para manipulação de dados e algoritmos do *scikit-learn* como exemplos de aprendizado de máquina aplicado. O sistema foi projetado com base em princípios de design orientado a objetos, priorizando encapsulamento e modularidade para reutilização de código. Entre os componentes desenvolvidos, destacam-se gerenciadores de estado, controladores de interface e um gerenciador genérico de modelos capaz de integrar diferentes algoritmos para processamento de dados. Este *boilerplate* visa simplificar o desenvolvimento de ferramentas de análise de dados, oferecendo uma solução escalável e reutilizável para projetos acadêmicos e industriais.

57

58

Palavras-chave: Boilerplate; Análise de Dados; Visualização de Dados; Aprendizado de Máquina; Tomada de decisão.

Abstract

This paper describes the development of a boilerplate aimed at creating interactive applications and tools for data analysis and visualization. The developed software offers a modular and extensible infrastructure, integrating GTK for graphical interface, matplotlib for visualization, pandas for data manipulation and scikit-learn algorithms as examples of applied machine learning. The system was designed based on object-oriented design principles, prioritizing encapsulation and modularity for code reuse. Among the developed components, state managers, interface controllers and a generic model manager capable of integrating different algorithms for data processing stand out. This boilerplate aims to simplify the development of data analysis tools, offering a scalable and reusable solution for academic and industrial projects.

Keywords: Boilerplate; Data Analysis; Data Visualization; Machine Learning; Decision Making.

Sumário

73	Lista de Figuras	iii
74	Lista de Códigos	v
75	Glossário	vii
76	1 Introdução	1
77	1.1 Motivação	1
78	1.2 Objetivos	2
79	1.2.1 Objetivos Específicos	2
80	1.3 Contribuições	2
81	2 Fundamentação Teórica	5
82	2.1 <i>Boilerplate</i>	5
83	2.2 Análise e visualização de dados	6
84	2.3 Aprendizado de Máquina	6
85	2.4 Linguagens de Programação	7
86	2.5 Bibliotecas	8
87	2.5.1 Numpy	8
88	2.5.2 Pandas	9
89	2.5.3 Scikit-learn	9
90	2.5.4 Seaborn/Matplotlib	9
91	2.5.5 GTK	9
92	2.6 Metodologia	11
93	2.6.1 Levantamento de Requisitos	11
94	3 Desenvolvimento	13
95	3.1 Arquitetura	14
96	3.1.1 Instalação e Configuração	14
97	3.1.2 Estrutura de Arquivos e Diretórios	15
98	3.2 Observador e Estado	18
99	3.2.1 Observador	19
100	3.2.2 Estado	21
101	3.3 Controller e Module	26
102	3.3.1 Controller	27
103	3.3.2 Module	28
104	3.4 Store	30
105	3.5 Banco de Dados	32

106	3.5.1	Definição de <i>Schemas</i> com <i>Pydantic</i>	33
107	3.5.2	Tabelas Tipadas	33
108	3.5.3	Configurações e Arquitetura	34
109	3.6	Gerenciamento de Modelos	34
110	3.6.1	Estrutura do Gerenciador de Modelos	34
111	3.6.2	Benefícios do Gerenciamento de Modelos	36
112	3.7	Interface Gráfica do <i>Boilerplate</i>	37
113	3.8	Utilitários	38
114	3.8.1	Logger	38
115	3.8.2	Documentação	39
116	3.8.3	Análise e Formatação de Código	40
117	3.8.4	Tradução e Internacionalização (i18n)	40
118	3.9	Considerações Finais	41
119	4	Conclusões	43
120	4.1	Resultados	43
121	4.2	Trabalhos Futuros	44
122		Referências Bibliográficas	48

123

Lista de Figuras

124	3.1	Estrutura de Arquivos e Diretórios do Projeto.	16
125	3.2	Diagrama Geral de Arquitetura.	17
126	3.3	Diagrama de classe dos padrões Estado e Observador.	18
127	3.4	Diagrama de sequência de interação <i>Widget</i> -Estado.	25
128	3.5	Diagrama de classe <i>Controller-Module</i>	26
129	3.6	Diagrama de Arquitetura de Componente.	31
130	3.7	Interface Gráfica do <i>Boilerplate</i>	37

131

Lista de Códigos

132	2.1	Exemplo de conexão de um sinal a um botão no GTK.	10
133	3.1	Exemplo da conexão do <i>signal clicked</i> de um <i>Widget</i> e um <i>handler</i>	24
134	3.2	Exemplo de implementação do Controller.	27
135	3.3	Exemplo de implementação do Module.	29
136	3.4	Pseudo-Implementação da classe <i>Store</i>	31
137	3.5	Exemplo de configuração de um banco de dados e uma tabela de configurações.	32
138	3.6	Exemplo de definição de <i>schemas</i> com <i>Pydantic</i>	33
139	3.7	Implementação do Modelo <i>KMeans</i>	35
140	3.8	Exemplos de uso do <i>logger</i>	38

Glossário

Acrônimos

143	FA		<i>Features Analyzer</i>
144	GUI		<i>Graphical User Interface</i>
145	GTK		<i>GIMP Toolkit</i>
146	CSV		<i>Comma Separated Values</i>
147	XLSX		<i>Excel Open XML Spreadsheet</i>
148	API		<i>Application Programming Interface</i>
149	SDK		<i>Software Development Kit</i>
150	PVM		<i>Python Virtual Machine</i>
151	JIT		<i>Just-In-Time</i>
152	SVG		<i>Scalable Vector Graphics</i>
153	JSON		<i>JavaScript Object Notation</i>
154	PO		<i>Portable Object (GNU Gettext)</i>
155	SQL		<i>Structured Query Language</i>
156	NoSQL		<i>Not Only SQL</i>
157	ORM		<i>Object-Relational Mapping</i>
158	MLP		<i>Multi-Layer Perceptron</i>
159	KMEANS		<i>K-Means Clustering</i>

Capítulo 1

Introdução

Neste capítulo objetiva-se apresentar uma visão geral sobre o trabalho desenvolvido, contextualizando o problema, justificando a sua relevância e destacando as contribuições do estudo. Para isso, ele está estruturado da seguinte forma:

Na **Seção 1.1**, é apresentada a motivação que embasa este trabalho. São discutidos os desafios enfrentados na área de análise e processamento de dados, bem como as oportunidades identificadas para a criação de ferramentas que facilitem e otimizem esse processo.

Na **Seção 1.2**, são descritos o objetivo geral e os objetivos específicos do trabalho. Nessa seção é esclarecido o propósito principal da pesquisa e detalha os passos necessários para alcançar os resultados esperados.

Por fim, na **Seção 1.3** são listadas as contribuições principais do trabalho, destacando os avanços proporcionados pela solução desenvolvida, tanto no contexto técnico quanto prático. Nela também são descritas as melhorias em termos de acessibilidade, usabilidade e inovação no fluxo de trabalho em análise de dados.

1.1 Motivação

O avanço exponencial dos sistemas computacionais resulta em um volume crescente de dados sendo gerados diariamente. Esse cenário cria não apenas desafios, mas também oportunidades para o desenvolvimento de técnicas inovadoras de análise e processamento de dados, com o intuito de transformar grandes quantidades de informações brutas em significativas e úteis ([Tarapanoff, 1995](#)).

Essas informações podem ser utilizadas em processos de tomada de decisão, nos quais a imensidão de dados muitas vezes ofusca a percepção dos usuários ([Han et al., 2012](#)). Tarefas como mineração de dados, classificação e reconhecimento de padrões são exemplos de abordagens capazes de extrair percepções valiosas de grandes volumes de dados.

Essas técnicas encontram aplicação em diversas áreas. Na medicina, por exemplo, é possível identificar doenças com base em exames laboratoriais; na engenharia, detectar falhas em equipamentos; e na segurança, identificar atividades maliciosas em redes de computadores. No entanto, para que essas tarefas sejam realizadas de maneira eficiente, é fundamental que os dados sejam coletados, processados e analisados adequadamente.

No contexto da análise e processamento de dados, as ferramentas disponíveis são majoritariamente de código fechado, dificultando a customização e a integração com outras soluções. Além disso, muitas dessas ferramentas são pagas, tornando seu uso inviável para

194 pesquisadores e pequenas empresas, que frequentemente possuem recursos financeiros li-
195 mitados.

196 Para a utilização de ferramentas de código aberto, muitas vezes é necessário conheci-
197 mento técnico avançado, criação de *scripts* e configurações manuais, o que pode ser um
198 obstáculo para usuários menos experientes, além de demandar tempo e esforço.

199 Diante desse contexto, surgiu a oportunidade de desenvolver uma ferramenta de código
200 aberto que seja personalizável e permita a integração com implementações já existentes
201 de algoritmos de processamento de dados. A ferramenta inclui uma interface gráfica
202 para interação, ajuste de parâmetros e visualização dos resultados, e tem como finalidade
203 proporcionar aos usuários a possibilidade de concentrar-se na resolução de seus problemas
204 específicos, sem a necessidade de implementar rotinas básicas e repetitivas.

205 1.2 Objetivos

206 O objetivo geral deste trabalho é desenvolver uma ferramenta de código aberto para
207 análise e processamento de dados, que seja customizável e extensível, permitindo sua
208 adaptação de forma ágil às necessidades específicas de pesquisadores e desenvolvedores.

209 1.2.1 Objetivos Específicos

210 Para alcançar o objetivo geral, os seguintes objetivos específicos foram estabelecidos:

- 211 • Desenvolver uma interface gráfica intuitiva para a visualização de dados, com um
212 sistema de estados que facilite a interação do usuário;
- 213 • Implementar funcionalidades para importação e exportação de dados em múltiplos
214 formatos, garantindo flexibilidade na integração com outras ferramentas;
- 215 • Garantir suporte para a integração de qualquer tipo de algoritmo de análise, pré-
216 processamento e processamento de dados, com foco na compatibilidade com biblio-
217otecas externas;
- 218 • Projetar e implementar uma estrutura de gerenciamento de modelos para otimizar
219 o treinamento, a validação e a avaliação de algoritmos de processamento de dados e
220 aprendizado de máquina;
- 221 • Desenvolver um sistema de compartilhamento de configurações, permitindo que in-
222 terfaces personalizadas e modelos configurados sejam facilmente reutilizados e dis-
223 tribuídos;
- 224 • Criar uma interface de documentação integrada, que seja customizável com tutori-
225 ais e exemplos práticos, para auxiliar os usuários na utilização da ferramenta por
226 terceiros em grupos de pesquisa e desenvolvimento;

227 1.3 Contribuições

228 Este trabalho apresenta contribuições para as áreas de visualização, análise e proces-
229 samento de dados, atendendo tanto a aspectos técnicos quanto práticos. As principais
230 contribuições são:

- 231 • **Interface gráfica do Usuário (*GUI*):** Projeto e implementação de um *framework*
232 ou *boilerplate* baseado em GTK, que facilita a interação com usuários. A interface
233 permite a visualização de dados, ajuste de parâmetros e execução de algoritmos,
234 eliminando a necessidade de conhecimentos avançados em programação de novas UI.
- 235 • **Otimização do fluxo de trabalho em análise de dados:** A ferramenta inte-
236 gra funcionalidades como importação e exportação de dados em múltiplos formatos,
237 gerenciamento de modelos e compartilhamento de configurações, promovendo maior
238 eficiência e flexibilidade para equipes de trabalho ou pesquisa. Além disso, dispo-
239 nibiliza uma interface de gerenciamento de modelos com suporte a armazenamento
240 dos parâmetros e reutilização de modelos.
- 241 • **Contribuição para a comunidade científica e de desenvolvedores:** Como
242 um projeto de código aberto, a ferramenta fomenta a colaboração e a inovação,
243 permitindo que pesquisadores e desenvolvedores adicionem novas funcionalidades e
244 adaptem a aplicação a problemas específicos.
- 245 • **Documentação integrada e colaborativa:** Implementação de uma interface de
246 documentação customizável, que permite registrar e compartilhar experimentos re-
247 alizados. Essa funcionalidade facilita a replicação de resultados e promove a disse-
248 minação de conhecimento entre grupos de pesquisa e desenvolvimento.
- 249 • **Base para projetos futuros:** Com uma arquitetura modular, a ferramenta é
250 projetada para suportar expansões e ser aplicada em diferentes domínios. Além de
251 atender à análise de dados, sua estrutura inicial também pode ser adaptada para o
252 desenvolvimento de interfaces gráficas em outros contextos.

Capítulo 2

Fundamentação Teórica

Neste capítulo são apresentados os conceitos e as definições necessárias para o entendimento do trabalho proposto. São abordados conceitos de *boilerplate*, análise e visualização de dados, aprendizado de máquina, linguagens de programação e bibliotecas. Além disso, é apresentada a metodologia utilizada para o desenvolvimento do trabalho.

2.1 *Boilerplate*

O termo *boilerplate* refere-se a blocos de código reutilizáveis que fornecem uma estrutura inicial ou funcionalidade básica para o desenvolvimento de sistemas ou aplicações (Lämmel e Jones, 2003). Esse conceito é amplamente utilizado na engenharia de software para acelerar o processo de desenvolvimento, padronizar práticas de codificação e minimizar erros comuns.

Na prática, um *boilerplate* atua como um ponto de partida que contém configurações, dependências e módulos essenciais já pré-configurados, permitindo que os desenvolvedores se concentrem mais na lógica específica da aplicação ao invés de investir tempo na configuração inicial do ambiente.

Um *boilerplate* geralmente inclui:

- Estrutura de diretórios organizada e padronizada.
- Arquivos de configuração para ferramentas como gerenciadores de pacotes, servidores ou *frameworks*.
- Código base com funcionalidades genéricas, como autenticação, conexão a bancos de dados, ou interface de usuário básica.
- Dependências e bibliotecas pré-definidas, integradas ao projeto.

Uma distinção importante no desenvolvimento de software é a diferença entre bibliotecas e *frameworks*. Enquanto bibliotecas são coleções de funções ou classes reutilizáveis que os desenvolvedores podem chamar diretamente, *frameworks* definem uma estrutura mais rígida para o desenvolvimento, fornecendo não apenas ferramentas, mas também controlando o fluxo da aplicação. Essa característica torna os *frameworks* menos flexíveis em comparação às bibliotecas, ao determinarem como o desenvolvedor deve construir a aplicação.

A escolha de utilizar um *boilerplate* neste projeto está diretamente relacionada à flexibilidade que ele oferece. Diferentemente de um *framework*, o *boilerplate* não impõe restrições rígidas ao fluxo ou à arquitetura da aplicação, permitindo que o desenvolvedor mantenha maior controle sobre o design e a lógica do sistema. Além disso, ele possibilita a edição do código base, e a integração de diferentes bibliotecas e componentes de maneira modular e adaptável, atendendo às necessidades específicas de cada projeto.

Neste trabalho, o conceito de *boilerplate* será aplicado ao desenvolvimento de uma ferramenta ou aplicação para análise e visualização de dados, oferecendo uma base sólida que pode ser facilmente ajustada a diferentes cenários e demandas. Essa abordagem combina a padronização inicial com a liberdade de personalização, tornando-a ideal para projetos acadêmicos e industriais.

2.2 Análise e visualização de dados

A análise de dados é uma tarefa complexa, mas de fundamental importância para a tomada de decisões (Guimarães e Évora, 2004). Esse processo pode ser descrito como a transformação de dados brutos em informações valiosas e significativas.

Com o aumento exponencial da produção e coleta de dados, impulsionado pelo advento da Internet das Coisas (IoT) e pela atividade dos próprios usuários, a análise de dados tornou-se essencial para agregar valor a evolução contínua da Web e das tecnologias associadas.

Neste contexto, a visualização de dados desempenha um papel crucial nesse processo, por ser através dela que o usuário pode interpretar os dados, identificar necessidades de pré-processamento, selecionar as técnicas de processamento adequadas e avaliar a qualidade dos resultados obtidos.

- **Pré-processamento:** inclui tarefas como formatação, remoção de dados faltantes, remoção de valores atípicos (*Outliers*), normalização, entre outros. Por exemplo, interpolação do valor anterior e posterior a um dado ausente com o objetivo de preencher essa lacuna em uma série temporal com baixa variabilidade.
- **Processamento:** pode ser realizada de diversas formas, tal como, a aplicação de métodos estatísticos ou algoritmos de aprendizado de máquina. Por exemplo: regressão, classificação, agrupamento, predição, otimização, entre outros.
- **Pós-processamento:** consiste na apresentação dos resultados obtidos, que pode ser por simples tabelas, gráficos, modelos matemáticos ou até valores numéricos. Por exemplo: visualização de dados em gráficos de séries históricas.

2.3 Aprendizado de Máquina

Dentre as várias técnicas utilizadas na análise de dados, o aprendizado de máquina consiste em um conjunto de algoritmos projetados para resolver problemas nos quais os métodos clássicos não são eficazes ou eficientes (Monard e Baranauskas, 2003). Esses algoritmos são capazes de gerar modelos matemáticos com base em dados, os quais podem ser utilizados para prever comportamentos, agrupar informações ou classificá-las.

Os problemas abordados por esses algoritmos podem ser divididos em diversas categorias, como, por exemplo: regressão, classificação, agrupamento, predição, detecção de anomalias, ranqueamento, otimização, entre outros.

- **Regressão:** Trata-se de uma técnica que visa modelar a relação entre uma variável dependente (ou resposta) e uma ou mais variáveis independentes (ou preditoras). O objetivo é encontrar uma função matemática que descreva essa relação com certa precisão. Uma vez obtida, essa função pode ser utilizada para prever valores futuros da variável dependente com base em novos dados das variáveis independentes. Por exemplo, prever o preço de uma casa com base em características como tamanho, localização e número de quartos.
- **Classificação:** Refere-se ao processo de atribuir categorias ou rótulos pré-definidos aos dados com base em suas características. Isso é feito ao identificar uma função que relacione os dados às categorias, permitindo determinar a classe de novas entradas. Por exemplo, classificar mensagens de e-mail como *spam* ou *não spam*, ou categorizar imagens como contendo ou não um determinado objeto.
- **Agrupamento:** Trata-se de identificar grupos de dados que compartilhem características semelhantes, sem informações prévias sobre os grupos. Um exemplo seria agrupar clientes de uma loja com base em seus hábitos de compra.
- **Predição:** Refere-se à tarefa de estimar valores futuros com base em dados históricos. Por exemplo, prever o preço de ações no mercado financeiro.
- **Detecção de anomalias:** Envolve identificar dados que se desviam significativamente do padrão observado. Esses dados podem ser removidos ou analisados separadamente, como em sistemas de detecção de fraudes.
- **Ranqueamento:** Esta técnica ordena os dados de acordo com uma métrica de relevância. Um exemplo comum é o ranqueamento de resultados em mecanismos de busca.
- **Otimização:** Consiste em encontrar soluções que maximizem ou minimizem uma função objetivo. Por exemplo, determinar a rota mais eficiente em um problema de logística.

Cada técnica citada, e muitos outros, já estão implementados em bibliotecas de código, algumas de código aberto, como, por exemplo, o *scikit-learn*, e outras de código fechado, como, por exemplo, o MATLAB. Estas bibliotecas são utilizadas para estudo, aplicações práticas, criação de plataformas para a realização de análises de dados, dentre outras aplicações.

2.4 Linguagens de Programação

A linguagem escolhida para este projeto foi o Python. Presente entre as linguagens mais utilizadas no mundo, o Python possui uma vasta comunidade de desenvolvedores e um número crescente de bibliotecas, consolidando-se como uma ferramenta rica para tarefas de análise de dados. Trata-se de uma linguagem de alto nível, multiparadigma,

com tipagem dinâmica e forte (Van Rossum e Drake Jr, 1995a). Além disso, sua sintaxe simples e intuitiva facilita o aprendizado tanto para iniciantes quanto para programadores experientes (Van Rossum e Drake Jr, 1995b).

Especificamente para este projeto, o Python foi utilizado com anotações de tipo, uma funcionalidade introduzida na versão 3.5 da linguagem. As anotações de tipo permitem definir os tipos de variáveis, argumentos e retornos de funções. Embora opcionais, elas são valiosas para a documentação do código e para a detecção precoce de erros. Ferramentas de análise estática, como o `mypy`, podem verificar se o código segue as definições de tipo, contribuindo para maior qualidade, facilidade de manutenção e extensão do projeto (Mypy, 2024).

Sobre o funcionamento, o Python executa sobre uma máquina virtual, chamada de Python Virtual Machine (PVM), que interpreta o código-fonte e executa as instruções (Ike-Nwosu, 2017). Ele utiliza uma abordagem de compilação *just-in-time* (JIT), na qual o código é transformado em *bytecode* e, posteriormente, em código de máquina durante a execução (Ike-Nwosu, 2017).

Embora o Python apresente desempenho inferior em relação a linguagens como C e C++, seu principal diferencial reside na ampla gama de bibliotecas e *frameworks* disponíveis, além de sua curva de aprendizado mais acessível. Este é concessão *trade-off* que justifica sua escolha para este projeto.

Outra razão relevante para a escolha do Python foi sua praticidade na construção de interfaces gráficas, especialmente em comparação com linguagens como C e C++. O Python disponibiliza bibliotecas como `PyGObject`, `PyQt`, `Tkinter` e `Kivy`, que tornam o desenvolvimento de interfaces interativas mais ágil e acessível, além de facilitar a criação de designs visualmente atraentes.

Em contrapartida, a criação de interfaces gráficas mais robustas frequentemente exige um volume significativo de código para gerenciar eventos e widgets. Já com Python, grande parte dessa complexidade é abstraída, permitindo ao desenvolvedor concentrar-se nas funcionalidades centrais da aplicação. Por fim, devido à sua popularidade, comunidade ativa, riqueza de bibliotecas, suporte para interfaces gráficas e anotações de tipo, o Python foi escolhido como a base de desenvolvimento deste projeto.

2.5 Bibliotecas

Bibliotecas são conjuntos de códigos que podem ser importados para outros projetos, proporcionando a reutilização de rotinas já implementadas para realizar tarefas específicas. A seguir, são apresentadas as principais bibliotecas utilizadas neste projeto:

2.5.1 Numpy

A `numpy` é uma biblioteca fundamental para a computação científica. Ela implementa vetores n-dimensionais em C, sendo compilados e executados nativamente, garantindo alto desempenho (Harris et al., 2020). Além disso, o `numpy` oferece uma vasta gama de funções matemáticas e operações de álgebra linear. Por exemplo, é possível normalizar um conjunto de dados com apenas uma linha de código, demonstrando sua eficiência e praticidade.

403 2.5.2 Pandas

404 A **pandas** é uma biblioteca de código aberto que utiliza o **numpy** como base para
405 estruturar os dados em tabelas chamadas *DataFrames*, estruturas multidimensionais am-
406 plamente empregadas na análise de dados ([pandas development team, 2020](#)). Por herdar
407 o desempenho do **numpy** e incluir uma ampla gama de funções específicas, a **pandas** per-
408 mite realizar operações como ordenação, filtragem e transformação de dados de maneira
409 eficiente e simplificada.

410 2.5.3 Scikit-learn

411 A **scikit-learn** (ou **sklearn**) é uma biblioteca de *machine learning* que fornece
412 implementações de algoritmos de aprendizado de máquina, estatísticos e de mineração
413 de dados ([Pedregosa et al., 2011](#)). Com um conjunto abrangente de algoritmos para
414 classificação, regressão, agrupamento e redução de dimensionalidade, a **sklearn** também
415 inclui métricas de avaliação para medir o desempenho dos modelos criados. Por exemplo,
416 a criação de um modelo de árvore de decisão, utilizado para tarefas de classificação ou
417 regressão, é facilmente implementada com esta biblioteca.

418 2.5.4 Seaborn/Matplotlib

419 A **seaborn** é a biblioteca escolhida para a visualização de dados neste projeto. Baseada
420 no **matplotlib**, ela fornece interfaces de alto nível que simplificam a criação e persona-
421 lização de gráficos ([Waskom, 2024](#)). Gráficos como dispersão, linhas e barras podem ser
422 criados e estilizados rapidamente. Além disso, os gráficos gerados podem ser integrados à
423 interface gráfica desenvolvida com GTK, garantindo uma apresentação visual interativa.

424 2.5.5 GTK

425 O **GTK** (*GIMP Toolkit*) é um conjunto de ferramentas (*toolkit*) multiplataforma para a
426 criação de interfaces gráficas de usuário (GUI, do inglês, Graphical User Interface) ([GTK, 2024](#)).
427 Ele é amplamente utilizado em aplicações Linux e possui suporte para outras
428 plataformas, como Windows e macOS. A versão 3 do **GTK** foi escolhida para este projeto
429 devido à sua maturidade, estabilidade e ampla documentação.

430 Embora a versão 4 ofereça avanços em desempenho e novos recursos, como suporte apri-
431 morado para renderização com GPU, ela ainda apresenta um ecossistema menos maduro,
432 com bibliotecas auxiliares em processo de adaptação e menor disponibilidade de exemplos
433 e tutoriais. Por isso, a versão 3 foi considerada mais adequada para o desenvolvimento
434 deste projeto, priorizando robustez e compatibilidade.

435 Widget

436 No **GTK**, um **Widget** é a unidade fundamental da interface gráfica. Ele representa
437 um componente visual ou interativo da aplicação, como botões, campos de texto, caixas
438 de seleção, entre outros. Widgets são os blocos básicos que compõem a interface gráfica,
439 permitindo interações entre o usuário e a aplicação.

440 Cada *Widget* possui propriedades e comportamentos específicos, podendo ser configu-
441 rado para atender a diferentes necessidades da interface. Por exemplo, um botão pode ser

utilizado para executar ações ao ser clicado, enquanto uma entrada de texto permite ao usuário digitar informações.

A comunicação entre o componente visual e o código é realizada através dos **Signals**, um mecanismo essencial no GTK para lidar com eventos. Eles permitem que os widgets comuniquem mudanças ou ações para a aplicação. Por exemplo, quando um botão é clicado, ele emite um *signal* chamado "clicked". No código, é possível conectar funções específicas a esses sinais, de forma que a aplicação saiba o que fazer quando um evento ocorre.

No **PyGObject**, os sinais podem ser conectados diretamente aos widgets usando o método `connect`, como no exemplo abaixo 2.1.

```
button.connect("clicked", on_button_clicked)
```

Código 2.1: Exemplo de conexão de um sinal a um botão no GTK.

Fonte: O Próprio Autor (2025)

Neste exemplo, a função `on_button_clicked` será chamada sempre que o botão for clicado. Isso demonstra como os widgets e os sinais trabalham juntos para criar interatividade e dinamismo na interface gráfica.

PyGObject, GObject Introspection e Stubs

O **PyGObject** é o vinculador (*binding*) oficial para o GTK em Python, permitindo o uso das bibliotecas GTK e de outras partes do GNOME diretamente na linguagem Python ([PyGObject, 2024](#)). O termo *binding* refere-se à técnica de criar uma ponte entre bibliotecas escritas em uma linguagem de baixo nível, como **C**, e linguagens de mais alto nível, como Python. Com isso, é possível utilizar funções e objetos dessas bibliotecas nativas sem a necessidade de reescrever o código em Python.

O **PyGObject** utiliza o sistema *GObject Introspection*, uma infraestrutura que permite o acesso dinâmico às APIs de bibliotecas escritas em **C** ([Introspection, 2024](#)). O **GObject Introspection** gera metadados das bibliotecas em tempo de execução, possibilitando a interação com essas APIs a partir de linguagens de alto nível, como Python. Isso reduz a necessidade de criar *bindings* manuais, simplificando a integração de bibliotecas nativas, além de promover maior flexibilidade e eficiência no processo de desenvolvimento.

Os *stubs*, por sua vez, são arquivos auxiliares que descrevem as funções, classes, e tipos disponíveis em uma biblioteca. Esses arquivos fornecem informações detalhadas para ferramentas de desenvolvimento, permitindo suporte a autocompletar, verificação de tipos e documentação direta no ambiente de codificação. No contexto do **PyGObject**, os *stubs* ajudam os desenvolvedores a explorar as interface de Programação de Aplicações (APIs, do inglês, Application Programming Interface) do **GTK** de maneira mais eficiente, minimizando erros e agilizando o processo de desenvolvimento.

Por exemplo, ao utilizar o **PyGObject** e os *stubs*, é possível criar interfaces gráficas interativas com um código significativamente mais conciso e legível em comparação ao **C**. Além disso, essa abordagem elimina a necessidade de gerenciar detalhes complexos, como a alocação de memória, que seria necessária ao desenvolver GUIs diretamente com **GTK** em **C**.

2.6 Metodologia

A metodologia adotada para o desenvolvimento deste trabalho busca atender aos objetivos propostos, garantindo a qualidade e a eficiência do processo. Para isso, foi adotado um modelo de desenvolvimento iterativo e incremental, baseado no levantamento de requisitos descrito na seção 2.6.1.

2.6.1 Levantamento de Requisitos

Foi realizada uma análise para identificar os requisitos funcionais e não funcionais do sistema. Os requisitos principais incluem:

- Interface simples, intuitiva e responsiva que permita fácil navegação entre as funcionalidades.
- Arquitetura de fluxo de interações baseada nos padrões de projeto de Estado e Observador.
- Importação de dados em formatos amplamente utilizados, como CSV e XLSX.
- Visualização interativa de dados por meio de gráficos.
- Suporte a integração de algoritmos de análise de dados.
- Banco de dados dos parâmetros dos modelos treinados em JSON para simplificar a visualização e reutilização.
- Exportação e carregamentos de modelos já treinados por meio de arquivos.
- Suporte a múltiplos sistemas operacionais, com foco inicial em Linux.
- Ferramenta para documentação externa e interna da ferramenta, permitindo que cada extensão ou experimento realizado na plataforma seja documentado.

Capítulo 3

Desenvolvimento

Neste capítulo é apresentada uma breve descrição do *boilerplate* do sistema, destacando os principais aspectos da arquitetura e implementação.

A Seção 3.1 apresenta a estrutura do projeto, a partir da instalação e configuração, destacando a organização dos arquivos, utilitários, módulos e suas respectivas responsabilidades. A proposição de uma estrutura bem definida é fundamental para a manutenção e evolução do sistema, além de facilitar o desenvolvimento de novas funcionalidades.

Na Seção 3.2, são detalhados os fundamentos da estrutura de interação com a interface gráfica, que utiliza principalmente dois padrões de projeto: *Observer* e *State*. Esses padrões definem como as interações entre a interface gráfica e os estados de contexto da aplicação são realizadas, garantindo modularidade e eficiência.

A Seção 3.3 são descritas as camadas de *controller* e *module*, que desempenham, respectivamente, o papel de conectar as ações dos *widgets* e assegurar a propagação dos estados para o restante da aplicação.

A descrição do contexto global de estados pode ser observada na Seção 3.4, que apresenta o módulo de gerenciamento de estados, que centraliza as operações relacionadas e fornece uma interface para salvar e restaurar o estado da aplicação. Esse módulo simplifica a integração dos estados com a interface gráfica, fornecendo uma forma eficiente de reutilização da aplicação, ao permitir que o estado seja salvo e restaurado em cada execução.

Na Seção 3.5 é apresentado o módulo de banco de dados, que fornece uma interface simples para salvar e carregar dados eficientemente. Esse módulo fornece uma camada de abstração para a manipulação de dados, permitindo salvar informações como estados, configurações, resultados, parâmetros de modelos e outros dados relevantes para a aplicação.

Os modelos de pré-processamento, processamento e avaliação são apresentados na Seção 3.6, destacando as principais características e funcionalidades do gerenciador de modelos. Esses modelos são essenciais para a execução das rotinas de análise de dados, fornecendo interfaces para salvar, carregar e executar modelos treinados.

A interface gráfica criada como exemplo para o *boilerplate* é descrita na Seção 3.7, destacando as principais funcionalidades e elementos visuais. Essa interface é um exemplo de como os conceitos e padrões de projeto apresentados podem ser aplicados na prática, fornecendo uma base sólida para o desenvolvimento de aplicações mais complexas.

O conjunto de utilitários é descrito na Seção 3.8, que fornece uma série de funções e ferramentas para facilitar o desenvolvimento de aplicações. Esses utilitários incluem funções para auditoria, verificação de código, documentação e internacionalização.

Para rever todos os aspectos do desenvolvimento e uso da aplicação desenvolvida, na Seção 3.9 é apresentado um resumo das principais funcionalidades e características do sistema, destacando os principais pontos abordados e sua relevância no contexto geral do trabalho.

3.1 Arquitetura

Nesta seção, é apresentada a instalação, configuração do projeto e execução, destacando as dependências e ferramentas utilizadas. Em seguida, é descrita a estrutura de arquivos e diretórios do projeto, com ênfase nos principais elementos. Por fim, é apresentada a arquitetura geral do projeto, destacando os principais componentes e suas interações.

3.1.1 Instalação e Configuração

Para melhor compreensão da arquitetura do projeto, é importante conhecer as dependências e ferramentas utilizadas. Para auxiliar no entendimento deste capítulo, a instalação e configuração do projeto são descritas nesta Subseção 3.1.1.

Importação

Para obter acesso ao projeto é possível importar a partir do **GitHub** por meio do comando `git clone git@github.com:ZauJulio/FeaturesAnalyzer.git` ou baixando o arquivo compactado diretamente do repositório. A *branch* `test` contém a versão utilizada neste trabalho, que pode ser acessada com o comando `git checkout test`.

Após a importação, é necessário instalar as dependências do projeto, que estão listadas no arquivo [pyproject.toml](https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/pyproject.toml)^[i] e [requirements.txt](https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/requirements.txt)^[ii].

Instalação

Para instalar as dependências do projeto, é recomendado o uso do [poetry](https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/pyproject.toml) com [pyenv](https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/pyproject.toml), que facilita a gestão de dependências e ambientes virtuais. O `poetry` utiliza os arquivos `pyproject.toml` e `poetry.lock` para registrar as configurações e dependências do projeto, garantindo a reprodutibilidade do ambiente em diferentes sistemas e colaboradores.

Com o `poetry` instalado, as dependências podem ser instaladas com o comando `make prepare`. Caso o comando `make` não esteja disponível, é possível instalar de acordo com cada distribuição.

- Debian/Raspbian/Ubuntu/Kali: `apt-get install make`
- Alpine: `apk add make`
- Arch: `Linux pacman -S make`
- CentOS: `yum install make`
- Fedora: `dnf install make`
- Windows: (WSL2)

^[i]<https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/pyproject.toml>

^[ii]<https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/requirements.txt>

573 • OS X: `brew install make`

574 Caso o `poetry` não esteja disponível, as dependências podem ser instaladas manual-
575 mente com o `pip`. Para isso, basta executar o comando `pip install -r requirements.txt`
576 na raiz do projeto.

577 Configuração

578 Além das dependências, o projeto conta com arquivos de configuração na raiz, como
579 o `.env.example`, `mkdocs.yml` e `ruff.toml`, que facilitam o gerenciamento do ambiente.
580 O arquivo `.env.example`^[iii] contém variáveis de ambiente que devem ser configuradas con-
581 forme as necessidades do projeto em um novo arquivo `.env`.

582 Execução

583 Após a instalação e configuração do projeto, a execução pode ser feita de maneira
584 simplificada com o comando `make run`. Além da instalação e execução, o `Makefile` conta
585 com outros comandos úteis, como `lint`, `serve` e `migrate-deps` para facilitar o desenvol-
586 vimento e manutenção do projeto. Para mais informações, consulte o arquivo `Makefile`^[iv].

587 Uma vez que o projeto foi importado, as dependências instaladas e as configurações
588 realizadas, é possível acompanhar o desenvolvimento deste capítulo com a execução do
589 projeto.

590 3.1.2 Estrutura de Arquivos e Diretórios

591 A descrição da arquitetura inicia-se pela organização da estrutura de arquivos do pro-
592 jeto, destacando os diretórios e arquivos mais relevantes. Esta estrutura foi planejada para
593 garantir a modularidade, clareza e facilidade de manutenção do código. Na Figura 3.1 é
594 apresentada a estrutura de arquivos e diretórios do projeto, detalhada na Lista 3.1.2, e na
595 Figura 3.2 sua arquitetura geral, que será discutida em detalhes nas próximas seções.

^[iii]<https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/.env.example>

^[iv]<https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/Makefile>

src	
├── assets	ARQUIVOS DE RECURSOS DO PROJETO
│ ├── styles	ESTILOS GLOBAIS
│ └── icons	ÍCONES
├── docs	DOCUMENTAÇÃO USADA PELO MkDocs
├── i18n	ARQUIVOS DE INTERNACIONALIZAÇÃO
├── src	CÓDIGO-FONTE DO PROJETO
│ ├── context	ESTADO GLOBAL
│ │ ├── states/*	ESTADOS COM MANIPULADORES
│ │ └── store.py	GERENCIADOR DE CONTEXTO
│ ├── models/*	MODELOS DO PROJETO, BASEADOS EM FAmODEL
│ ├── interfaces	INTERFACES DO PROJETO
│ │ ├── application.py	INTERFACE GLOBAL DA APLICAÇÃO
│ │ ├── controller.py	MANIPULADOR GENÉRICO E CONEXÃO DE SINAIS
│ │ ├── module.py	MANIPULADOR DE SINAIS DE ESTADO RAIZ
│ │ ├── state.py	ESTADO GENÉRICO
│ │ └── widget.py	WIDGET GENÉRICO, ESTENDE GTK.WIDGET
│ └── lib	BIBLIOTECAS DO PROJETO
│ ├── model_manager	GERENCIADOR DE MODELOS
│ ├── orm	ORM TIPADO COM PYDANTIC E TINYDB
│ ├── state_manager	GERENCIADOR DE ESTADOS COM OBSERVER
│ ├── utils	UTILITÁRIOS GENÉRICOS
│ │ ├── bin.py	ACESSO A MODELOS BINÁRIOS
│ │ ├── hash.py	UTILITÁRIOS DE HASH
│ │ ├── lock.py	UTILITÁRIOS PARA MULTIPROCESSAMENTO
│ │ ├── logger.py	LOGGER GLOBAL
│ │ ├── meta.py	UTILITÁRIOS DE <i>metaclasses</i> , COMO <i>Singleton</i>
│ │ ├── time.py	FUSOS HORÁRIOS, ETC.
│ │ ├── types.py	TIPOS PARA O GERENCIADOR DE ESTADOS
│ │ └── ui.py	UTILITÁRIOS DE INTERFACE GRÁFICA
│ └── widget.py	WIDGET GENÉRICO, ESTENDE GTK.WIDGET
└── sdk	SDKs PARA OUTROS SERVIÇOS
├── .env.example	EXEMPLO DE VARIÁVEIS DE AMBIENTE
├── Makefile	MAKEFILE COM COMANDOS COMUNS
├── pyproject.toml	CONFIGURAÇÃO DO PROJETO PYTHON
└── requirements.txt	DEPENDÊNCIAS PYTHON (ALTERNATIVA AO POETRY)

Figura 3.1: Estrutura de Arquivos e Diretórios do Projeto.

Fonte: O Próprio Autor (2025)

596 Descrição Geral

597 A estrutura de arquivos segue boas práticas de organização, com destaque para:

- 598 • **assets**: Armazena arquivos de recursos visuais, como estilos e ícones.
- 599 • **docs**: Contém a documentação do projeto, que pode ser gerada com ferramentas
- 600 como o MkDocs.

- **i18n**: Centraliza os arquivos de internacionalização, permitindo suporte a múltiplos idiomas.
- **src**: Diretório principal do código-fonte, subdividido em módulos funcionais, como `context`, `models` e `lib`.
 - **context**: Inclui o estado global da aplicação, com os estados e manipuladores associados.
 - **models**: Contém os modelos do projeto, baseados na classe `FAModel`.
 - **interfaces**: Define as interfaces do projeto, como `Application` e `Controller`.
 - **lib**: Agrupa as bibliotecas do projeto, organizadas em subdiretórios, como gerenciadores de modelos e utilitários gerais.
- Arquivos de configuração (`.env.example`, `Dockerfile`, `Makefile`, etc.) estão localizados na raiz, facilitando a automação e o gerenciamento do ambiente.

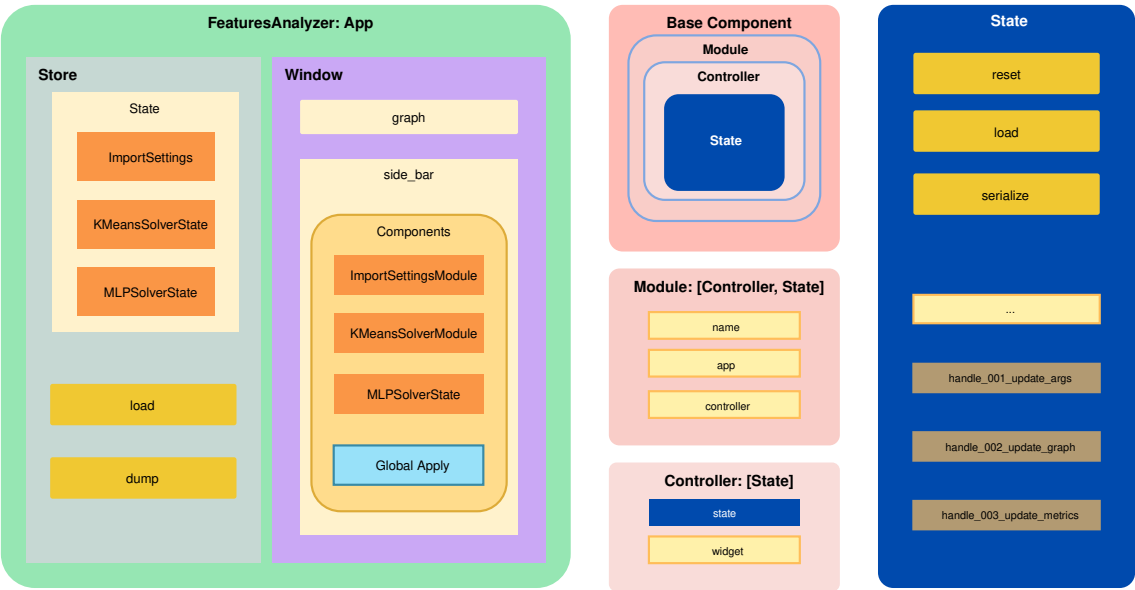


Figura 3.2: Diagrama Geral de Arquitetura.
Fonte: O Próprio Autor (2025)

Nesta Seção 3.2 é detalhado a implementação do componente base, que fornece a infraestrutura para a interface gráfica e o gerenciamento de estados.

^[v]Durante o desenvolvimento do sistema, o projeto recebeu o nome provisório de *Features Analyzer* (do inglês, Analisador de Características) (FA). Por essa razão, a sigla *FA* pode ser encontrada em diversos módulos e classes, sendo utilizada para distinguir os componentes do *boilerplate* daqueles específicos da aplicação que o utiliza. Essa abordagem contribui para uma separação clara entre a estrutura base e as personalizações realizadas para cada sistema desenvolvido.

616

3.2 Observador e Estado

617

618

619

A base do *boilerplate* da aplicação é construída com a associação de dois padrões de projeto fundamentais: *Observer* (Observador) e *State* (Estado). Essa seção é ilustrada na Figura 3.2 caixa azul, posicionada a direita, com título **State**.

620

621

622

623

O padrão *Observer* é utilizado para notificar funções de *callback* (funções que executam rotinas específicas utilizadas como parâmetro) quando um determinado contexto é alterado. Já o padrão *State* é empregado para o armazenamento de informações, além de possibilitar a criação de *getters* e *setters* para manipulação eficiente do estado.

624

625

626

627

628

Ao associar esses dois padrões a um *Widget*, é possível salvar e manipular o estado de um componente de maneira eficiente, utilizando *handler's* (funções responsáveis por tratar objetos específicos) para gerenciar o estado, e notificações de mudanças por meio de *callback's* fornecidos ao *Observer*. O diagrama de classe ilustrado na Figura 3.3 demonstra a relação entre as implementações dos padrões de projeto *Observer* e *State*.

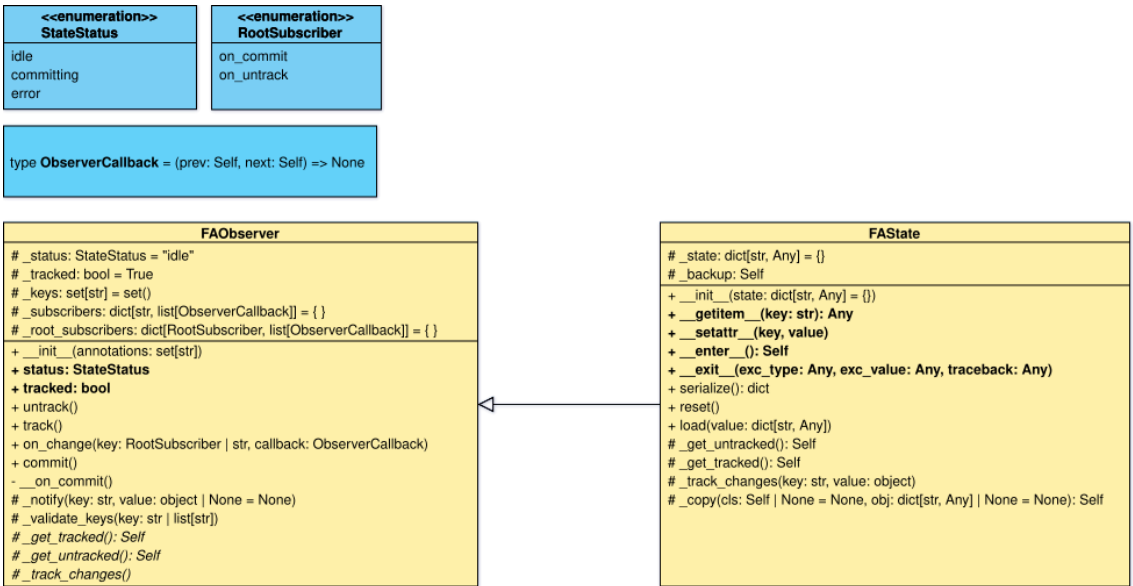


Figura 3.3: Diagrama de classe dos padrões Estado e Observador.
Fonte: O Próprio Autor (2025)

629

630

631

A implementação desses padrões de projeto estende-se ao domínio do problema do sistema. Nesse contexto, o estado de um objeto pode assumir dois valores distintos simultaneamente: estado rastreado e estado não rastreado.

632

633

634

635

636

O estado rastreado é utilizado para armazenar o contexto concreto do objeto; no entanto, assim que ocorre uma alteração, ele se torna um estado não rastreado, pois as mudanças ainda não foram propagadas para as funções observadoras. Para fins de simplicidade, além das abstrações, os estados são dois dicionários, um para o estado rastreado e outro para o estado não rastreado.

637

638

639

640

O principal objetivo dessa abordagem é evitar a propagação desnecessária de mudanças no estado. Isso é particularmente útil em situações onde o estado do objeto pode ser alterado várias vezes em um curto intervalo de tempo, como durante a interação do usuário com a interface gráfica ao ajustar os parâmetros de um modelo.

641

Por exemplo, na implementação desenvolvida, um simples botão de salvar pode ser

642 usado para realizar um *commit* do estado, definindo-o como rastreado e propagando as
643 alterações para o restante do sistema.

644 Nas subseções seguintes, são apresentados os detalhes da implementação dos padrões
645 de projeto *Observer* e *State*, bem como sua utilização para viabilizar a interação eficiente
646 entre a interface gráfica e o módulo de controle.

647 3.2.1 Observador

648 Na implementação do padrão *Observer*, a classe abstrata **Observer** é responsável por
649 definir a interface de notificação de mudanças de estado. Essa classe é estendida pela classe
650 **State**, que armazena o estado do objeto observado e notifica os *handlers* de mudança de
651 estado.

652 O diagrama da classe **Observer** é apresentado na Figura 3.3. No diagrama, os atri-
653 butos protegidos são identificados pelo símbolo #, indicando que não podem ser acessados
654 diretamente. As operações públicas, acessíveis por qualquer classe que herde ou instancie
655 *Observer*, são indicadas com +, enquanto as operações privadas, restritas à própria classe,
656 começam com -. Abaixo, são descritos os atributos e operações da classe **Observer**.

657 Atributos

658 Todos os atributos da classe **Observer** são protegidos, garantindo que não sejam aces-
659 sados diretamente por outras classes. Seus principais atributos incluem:

- 660 • **_status**: Indica o estado atual do objeto observado. Os estados possíveis são:
 - 661 – **idle**: O objeto não está em uso por nenhuma operação.
 - 662 – **committing**: O objeto está sendo atualizado.
 - 663 – **error**: Ocorreu um erro durante a atualização.
- 664 • **_tracked**: Define se o estado do objeto está rastreado (**True**) ou não (**False**), ou
665 seja, não houve alterações desde o último **commit()**.
- 666 • **_keys**: Armazena as chaves de notificação de mudanças, representando os nomes dos
667 atributos do objeto observado.
- 668 • **_subscribers**: Um objeto que lista os *handlers* que serão notificados sobre mudanças
669 nos atributos observados, conforme as chaves definidas em **_keys**.
- 670 • **_root_subscribers**: Contém *handlers* associados às mudanças de contexto do ob-
671 jeto observado, utilizando as seguintes chaves:
 - 672 – **on_untrack**: Notifica quando o estado deixa de ser rastreado.
 - 673 – **on_commit**: Notifica quando o estado é rastreado por meio de um *commit()*.

674 Operações

675 As operações da classe **Observer** estão organizadas em públicas, protegidas e privadas,
676 conforme descrito abaixo:

677 **Operações públicas:**

- 678 • `__init__`: Inicializa os atributos da classe. Recebe como parâmetro `annotations`,
679 uma lista de chaves de notificação de mudanças (`subscriber keys`).
- 680 • `status`: Retorna o estado atual do objeto observado. Acessado como propriedade
681 (`property`) e somente leitura.
- 682 • `tracked`: Retorna se o estado está sendo rastreado. Propriedade somente leitura,
683 assim como `status`.
- 684 • `untrack`: Define `_tracked` como `False` e notifica `_root_subscribers` com a chave
685 `on_untrack`.
- 686 • `track`: Define `_tracked` como `True` e ajusta o estado para `idle`.
- 687 • `on_change`: Adiciona um *handler* de notificação. Recebe como parâmetros:
 - 688 – `key`: A chave do atributo observado ou de `_root_subscribers`.
 - 689 – `handler`: Função de *callback* que recebe os valores antigo e novo do estado.
- 690 • `commit`: Atualiza todo valor não rastreado, ou alterado, para o objeto rastreado.
691 Notifica `_root_subscribers` com a chave `on_commit` e, para cada atributo alterado,
692 notifica os `_subscribers` correspondentes.

693
694

695 Por exemplo, um objeto observado qualquer `{ a : 1 }`, possui duas instancias, uma
696 rastreada e outra não rastreada. Uma vez que o atributo `a` seja alterado para 2, os
697 *handlers* de `a` são notificados, ou seja, as funções de *callback* associadas a chave `a`
698 são executadas. As funções recebem os valores antigo e novo do estado, `{ a : 1 }` e
699 `{ a : 2 }` respectivamente, permitindo a execução de lógicas específicas para cada
700 atributo.

701 Para rastrear a mudança de estado, o método `commit` sincroniza o estado ras-
702 treado com o valor concreto do objeto, notificando os `_subscribers` de `a` e os
703 `_root_subscribers` de `on_commit`. Ao realizar um `commit`, o estado rastreado é
704 atualizado com o valor do estado não rastreado, logo, o atributo `a` do objeto rastre-
705 ado passa a ser 2.

706 **Operações privadas:**

- 707 • `__on_commit`: Define o estado como `committing` e notifica `_root_subscribers` com
708 a chave `on_commit` durante o gerenciamento de contexto com a cláusula `with`. Ao
709 final da operação, define o estado como `idle` através de `track()`.

710 **Operações protegidas:**

- 711 • `_notify`: Notifica `_subscribers` e `_root_subscribers` sobre mudanças no estado.
- 712 • `_validate_keys`: Valida as chaves de notificação recebidas.

- 713 • `_get_tracked`, `_get_untracked`, `_track_changes`: Métodos abstratos que devem ser
714 implementados para rastrear mudanças de estado.
- 715 – `_get_tracked`: Retorna o estado rastreado do objeto observado.
- 716 – `_get_untracked`: Retorna o estado não rastreado do objeto observado.
- 717 – `_track_changes`: Sincroniza o estado rastreado com o valor concreto do objeto.
718 Recebe como parâmetro o valor do estado não rastreado e a chave do atributo
719 alterado.

720 A nomenclatura adotada com `_` e `__` indica que os atributos ou métodos são protegidos,
721 ou privados, respectivamente, seguindo as convenções do Python. Embora seja possível
722 acessar diretamente tais elementos, a prática desaconselha esse uso, contribuindo para
723 uma melhor manutenção do código.

724 Exemplo de Implementação

725 Como exemplo, uma classe pode herdar de `Observer` e implementar os métodos abstra-
726 tos `_get_tracked`, `_get_untracked` e `_track_changes`. Essa classe gerencia dois atributos:
727 `t_state` (estado rastreado) e `ut_state` (estado não rastreado), que possuem as mesmas
728 chaves, como `a` e `b`.

729 Uma vez que um atributo é alterado e o método `untrack` é chamado, o estado de
730 `_tracked` é alterado para falso e os `_root_subscribers` são notificados com a chave `on_untrack`.
731 Quando o método `commit` é chamado, o estado de `_tracked` é alterado para verdadeiro e os
732 `_root_subscribers` são notificados com a chave `on_commit`. Além disso, os `_subscribers`
733 são notificados com a chave correspondente ao atributo que passou por alteração.

734 A classe `State`, que herda de `Observer`, foi criada para evitar duplicação de código,
735 implementando os métodos abstratos necessários para rastrear mudanças de estado de
736 forma automática e eficiente.

737 3.2.2 Estado

738 A classe `State` é responsável por armazenar o estado de um objeto e, por meio da
739 herança da classe `Observer`, notificar *handlers* sobre mudanças no estado. Além de her-
740 dar os atributos e métodos da classe `Observer`, a classe `State` implementa os métodos
741 abstratos `_get_tracked`, `_get_untracked` e `_track_changes`, permitindo a automação no
742 rastreamento e propagação de mudanças no estado.

743 O objetivo principal da classe `State` é simplificar a manipulação de objetos com es-
744 tados rastreáveis, permitindo que os atributos do estado e os manipuladores associados
745 sejam definidos diretamente na classe derivada. Além disso, ela oferece funcionalidades
746 adicionais para gerenciar, serializar e restaurar estados, otimizando sua integração em
747 sistemas dinâmicos.

748 Atributos

- 749 • `_state`: Um dicionário que armazena o estado rastreado do objeto observado. É
750 inicializado com os valores fornecidos no momento da criação da classe através das
751 anotações de tipo e dos valores iniciais passados como parâmetro.

- `_backup`: Uma cópia temporária do estado rastreado, utilizada para preservar o estado antes de modificações, geralmente em operações envolvendo gerenciamento de contexto.

Operações

As operações públicas da classe `State` incluem:

- `__init__`: Inicializa o estado rastreado do objeto com os valores fornecidos por parâmetro em um dicionário `State`.
- `__getitem__`, `__setattr__`, `__enter__`, `__exit__`, `serialize`, `reset`, `load`: Métodos especiais do Python que permitem acessar, atualizar e gerenciar o contexto de execução, respectivamente.
 - `__getitem__`: Permite acessar os atributos do estado como itens de um dicionário, utilizando a notação `state[key]`. Logo, o estado não rastreado é armazenado como atributo da classe que herdou de `State`, e os rastreados de `_state`.
 - `__setattr__`: Atualiza o valor de um atributo no estado não rastreado e dispara os manipuladores de notificação de mudança através da `key on_untrack`.
 - `__enter__`: Inicia o gerenciamento de contexto, armazenando uma cópia do estado atual em `_backup`.
 - `__exit__`: Finaliza o gerenciamento de contexto, restaurando o estado a partir de `_backup` em caso de erro.
- `serialize`: Retorna uma representação serializável do estado atual, ideal para armazenamento ou transmissão.
- `reset`: Restaura o estado rastreado para os valores padrão conforme o tipo, como 0 para inteiros e `''` para strings.
- `load`: Atualiza o estado rastreado com os valores fornecidos no parâmetro dicionário `value`. Especialmente útil para restaurar estados salvos anteriormente.

As operações protegidas da classe `State` incluem:

- `_get_untracked`: Retorna uma instância do estado não rastreado, útil para inspeção sem disparar notificações.
- `_get_tracked`: Retorna o estado rastreado atual.
- `_track_changes`: Realiza o rastreamento de mudanças em um atributo específico do estado, recebe como parâmetros a chave do atributo e o valor a ser atualizado.
- `_copy`: Retorna uma cópia de um estado fornecido como parâmetro ou de dicionário como parâmetro (`cls` or `obj`).

Uso Prático

A classe `State` oferece uma abordagem eficiente para gerenciar estados, integrando rastreamento automático de mudanças com notificação de *handlers*. Esse modelo é especialmente útil em sistemas que dependem de estados compartilhados ou precisam sincronizar alterações em diferentes componentes, como interfaces gráficas ou sistemas distribuídos.

Uma das vantagens dessa classe é a possibilidade de agrupar estados em objetos compostos, facilitando a organização e manutenção de sistemas complexos. Por exemplo, em um sistema com módulos distintos — como importação de dados, pré-processamento e treinamento de modelos — os estados de cada módulo podem ser encapsulados em um dicionário `Context` ou `Store`. Isso permite que cada módulo gerencie seu próprio estado de forma independente, evitando interferências entre eles e simplificando a manipulação global, através da centralização dos estados.

Embora seja tecnicamente viável compor estados aninhados (ou "estados de estados"), essa prática não é recomendada. Essa abordagem pode introduzir complexidade desnecessária e problemas de sincronização. Em vez disso, estados relacionados devem ser agrupados em um único objeto, promovendo uma interação mais simples e garantindo a consistência dos dados.

Uma recomendação importante é centralizar no domínio de cada estado os *handlers* que gerenciem mudanças específicas. Por exemplo, em um estado responsável por carregar um arquivo, o *handler* associado pode receber o *path* do arquivo e, após um `commit`, executar a lógica necessária para carregar o arquivo e atualizar um gráfico ou interface de usuário. Para manter a organização e clareza, é sugerido que cada *handler* seja implementado como um método estático da classe derivada de `State`, seguindo uma convenção de nomenclatura padrão `handle_{id_seq}_{name}`.

Nesta convenção, `id_seq` é um número sequencial único que identifica o *handler*, e `name` é uma descrição clara da função do *handler*. Por exemplo, um *handler* responsável por carregar arquivos poderia ser nomeado como `handle_001_load_file`. Essa nomenclatura também evita conflitos de nomes, entre estados e com a própria classe `State`.

Essa abordagem promove consistência, organização e facilita a manutenção de sistemas complexos que utilizam a classe `State`. A seguir é apresentado um exemplo simples e prático do fluxo de interação entre um *Widget* e um estado gerenciado pela classe `State`.

[vi]

Widgets e Elementos Visuais

A biblioteca `GTK` é utilizada para a construção de interfaces gráficas, permitindo a criação de *widgets* (elementos visuais) personalizados e facilitando a interação com o usuário. A classe `Widget` encapsula os elementos visuais e manipula as interações, utilizando a classe `State` para monitorar e notificar as mudanças de estado.

A comunicação entre o `Widget` e o estado ocorre via *handlers* associados a eventos específicos, como cliques em botões ou alterações de valor. Esses *handlers* são definidos em uma classe derivada de `State` e acionados pelo `Widget` em resposta a eventos do usuário. Os *widgets* notificam mudanças ou ações por meio de *signals* do `GTK`, os quais

[vi] Utiliza-se a convenção `id_seq` para identificar a ordem que o *handler* deve ser executado, e `name` para descrever a função do *handler*. Embora a nomenclatura seja opcional, ela promove a organização e facilita a manutenção de sistemas complexos, além de evitar conflitos de nomes entre estados, *handlers* e a própria classe `State`.

são conectados aos *handlers* correspondentes. Abaixo, são apresentados alguns exemplos de *widgets* e seus respectivos *signals*:

- **Button:** `clicked` - Notifica quando o botão é clicado.
- **Entry:** `changed` - Notifica quando o texto é alterado.
- **ComboBox:** `changed` - Notifica quando o item selecionado é alterado.
- **CheckButton:** `toggled` - Notifica quando o botão é alternado.

Quando o usuário interage com um *Widget*, o *signal* correspondente é acionado, invocando o *handler* associado. Por exemplo:

```

835     button.connect("clicked", self.on_button_clicked)           1
836                                                                2
837     def on_button_clicked(self, widget):                        3
838         self.state.handle_001_plot_data()                      4
839         self.state.commit()                                    5
840                                                                6

```

Código 3.1: Exemplo da conexão do *signal* `clicked` de um *Widget* e um *handler*.

Fonte: O Próprio Autor (2025)

No código 3.1, o *handler* `handle_001_plot_data` é executado quando o botão é clicado, realizando a lógica necessária para a plotagem dos dados. Após a execução do *handler*, o estado é atualizado através do método `commit`, propagando as mudanças para os *handlers* associados. Como o *handler* está inserido no domínio do estado, ele pode acessar informações não rastreadas diretamente com o método `get_untracked()` antes da execução do método `commit()`.

Exemplo de Interação Widget-Estado

O fluxograma da interação entre um *Widget* e um estado gerenciado pela classe **State** é ilustrado na Figura 3.4. Neste exemplo, adicionamos uma camada intermediária, o **Controller**, que gerencia a interação entre o *Widget* e o estado. O **Controller** é responsável por conectar os sinais do *Widget* aos *setters* do estado, garantindo a correta propagação das mudanças de estado. O **Controller** também pode conter lógica adicional, como validação de entrada ou formatação de dados, antes de atualizar o estado. A seguir, são descritos os passos do diagrama de sequência.

- O *signal* de um botão é conectado a um *handler* no **Controller**.
- O **Controller** assina o estado para receber notificações, `state.on_change("on_untrack", self.on_untrack)` e `state.on_change("on_commit", self.on_commit)`.
 - `on_untrack`: Adiciona um botão de aplicar alterações à interface do usuário.
 - `on_commit`: Atualiza a interface do usuário com o novo estado e oculta o botão de aplicar.
- O usuário interage com o *Widget*, acionando o *signal* `"clicked"`.

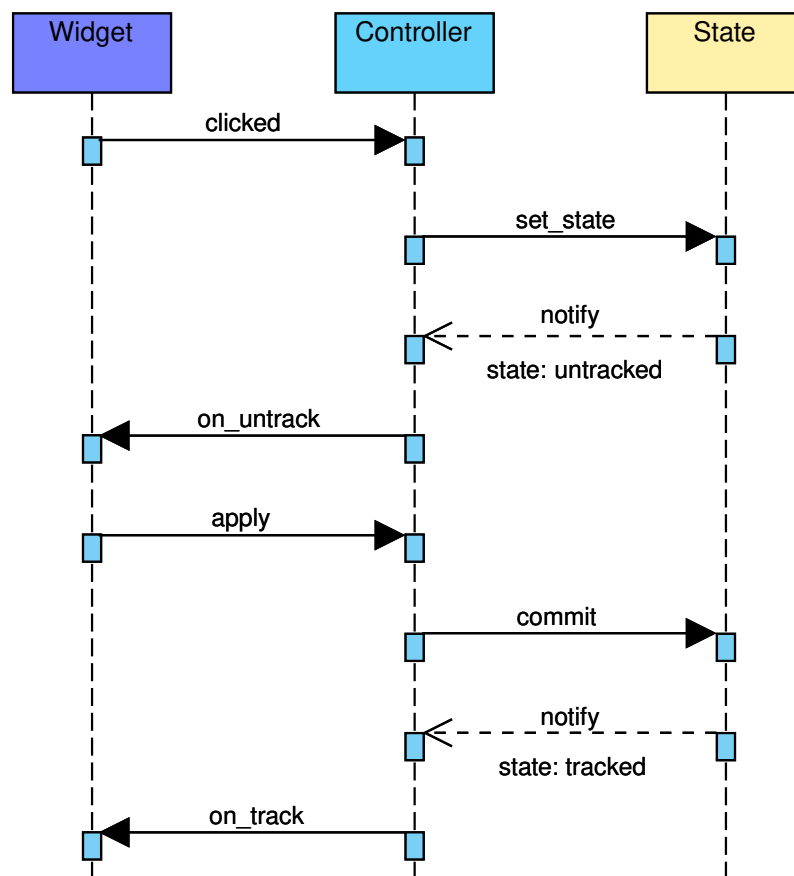


Figura 3.4: Diagrama de sequência de interação *Widget-Estado*.

Fonte: O Próprio Autor (2025)

- 862 • O **Controller** executa a lógica necessária, como validação de entrada ou formatação
863 de dados; após isso, define o valor no estado por meio de um *setter* simples, como
864 `state.value = value`.
- 865 • O **State** notifica os *handlers* associados à chave correspondente, como `on_value_change`
866 e `on_untrack`.
- 867 • O *handler* de `on_untrack` associado ao **Controller** executa a lógica necessária, e
868 adiciona a interface do usuário um botão para aplicar as alterações.
- 869 • O novo botão tem o *signal* conectado a um *handler* no **Controller**, que executa a
870 chamada para `state.commit()`.
- 871 • O usuário interage com o novo **Widget** do botão, acionando o *signal* "`clicked`".
- 872 • O **Controller** executa a chamada para `state.commit()`.
- 873 • O **State** notifica os *handlers* associados à chave correspondente, como `on_commit`.
- 874 • O **Controller** executa a lógica necessária, como atualizar a interface do usuário com
875 o novo estado e ocultar o botão de aplicar, já que o estado foi rastreado.

Nesta arquitetura o `Controller` atua como um intermediário entre o `Widget` e o estado, gerenciando a interação entre eles e garantindo a correta propagação das mudanças de estado. Essa abordagem promove a separação de responsabilidades e facilita a manutenção e evolução do sistema, tornando-o mais modular e flexível.

A camada de `Widget` é responsável pela interação com o usuário, definindo os elementos visuais, enquanto o `Controller` gerencia a lógica de execução e a comunicação com o estado. Desta forma, a lógica de validação, formatação e atualização de dados é centralizada no `Controller`, simplificando a implementação e garantindo a consistência dos dados.

Além da camada de `Controller`, podemos adicionar uma camada de `Module` para gerenciar alterações globais no estado, como aquelas notificadas pelas chaves `on_untrack` e `on_commit`. Na Seção 3.3 são apresentados os detalhes da implementação do módulo de controle e sua integração com a interface gráfica.

3.3 Controller e Module

A camada de `Controller` e `Module` desempenha o papel de separar as responsabilidades na propagação e manipulação dos estados, garantindo a consistência, manutenibilidade e escalabilidade do sistema. Na seção atual é apresentado o conteúdo previamente ilustrado na Figura 3.2, caixa do meio em vermelho, com título *Base Component*.

O `Controller` é responsável por gerenciar as ações dos *Widgets*, enquanto o *Module* lida com as mudanças de estado desencadeadas por essas ações no escopo do domínio de cada componente. A seguir, na Figura 3.5 é apresentado o diagrama de classes do `Controller` e `Module`.

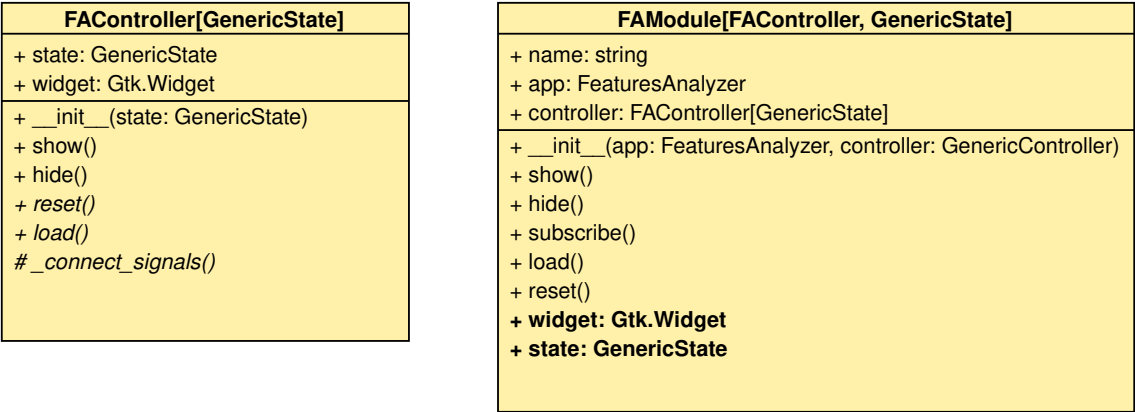


Figura 3.5: Diagrama de classe *Controller-Module*.
Fonte: O Próprio Autor (2025)

3.3.1 Controller

Nas subseções a seguir é apresentada a implementação da classe `Controller`.

Atributos

- **state**: Uma instância de `FASState` que armazena o estado do objeto observado. É utilizado para registrar os valores dos widgets e notificar os *handlers* de mudanças.
- **widget**: Uma instância de `Gtk.Widget` que representa o elemento visual associado ao `Controller`. É utilizado para conectar os sinais do widget aos *handlers* do `Controller`. Tipicamente pode ser um único botão ou um contêiner com vários elementos.

Operações

As operações da classe `Controller` incluem:

- **__init__**: Inicializa o `Controller` com uma instância de `State`, um `widget` associado é instanciado como parâmetro.
- **show**: Exibe o(s) `widget(s)` associado(s) ao `Controller`.
- **hide**: Oculta o(s) `widget(s)` associado(s) ao `Controller`.
- **_connect_signals**, **reset** e **load**: Métodos abstratos protegidos que devem ser implementados na classe derivada de `Controller`.
 - **_connect_signals**: Conecta os sinais do `widget` aos *handlers* do `Controller`.
 - **reset**: Restaura o estado do `widget` para os valores padrão.
 - **load**: Atualiza o estado do `widget` com os valores fornecidos no parâmetro `value` do tipo `State`.

No código 3.2 é apresentado um exemplo prático de implementação do `Controller` para um componente que controla os parâmetros de um modelo utilizando `KMeans`.

```

923 @FAMetaCheckController
924 class KMeansSolverController(FAController[KMeansSolverState]):
925     """KMeans Solver controller."""
926
927     state: KMeansSolverState
928     widget: KMeansSolverWidget = KMeansSolverWidget()
929
930     def __init__(self, state: KMeansSolverState) -> None:
931         self.state = state
932
933         self._connect_signals()
934
935         # Load the initial state
936         self.load(self.state)
937
938         if self.state.max_iter or self.state.n_clusters:
939             self.widget.on_module_change()

```

Código 3.2: Exemplo de implementação do `Controller`.

Fonte: O Próprio Autor (2025)

940

941

942 Na linha 8, é possível observar um decorador sobre a classe `@FAMetaCheckController`,
 943 responsável por verificar se os atributos necessários e seus respectivos tipos estão presentes
 944 na classe derivada. Esse decorador é uma abstração que simplifica a criação de novos
 945 *Controller's*, assegurando a consistência e integridade da implementação.

946 A linha 10 define a classe `KMeansSolverController`, que herda de `Controller` com o
 947 tipo `KMeansSolverState`. Essa sintaxe especifica que o atributo `State` é uma instância de
 948 `KMeansSolverState`. Além disso, na linha 13, é definida a instância do widget associado
 949 ao `Controller`, que pode ser encontrado no [arquivo^{\[vii\]}](#).

950 No método `__init__` de `KMeansSolverController`, iniciado na linha 15, observa-se
 951 a chamada do método `_connect_signals`, responsável por conectar os sinais do widget
 952 aos *handlers* do `Controller`. Já na linha 21, o método `load` é invocado para atualizar
 953 o estado do widget com os valores fornecidos no parâmetro `State`, o qual é do tipo
 954 `KMeansSolverState`. Por fim, caso algum valor tenha sido enviado através do parâmetro
 955 `State`, o widget adiciona, na linha 24, um botão para aplicar as alterações.

956 O exemplo apresentado demonstra a implementação de um `Controller`, evidenciando
 957 a simplicidade e a organização do código, que contribuem para a manutenção e evolução do
 958 sistema. Abstrações como o decorador `@FAMetaCheckController` e a classe `Controller`
 959 promovem consistência e robustez na implementação, assegurando a qualidade do sistema
 960 na totalidade.

961 3.3.2 Module

962 A seguir é apresentado o diagrama de classes do `Module` na Figura 3.5. O `Module`
 963 é responsável por gerenciar as mudanças de estado no escopo do domínio de cada com-
 964 ponente, assegurando a consistência e a propagação das alterações para o restante do
 965 sistema. Essa abstração encapsula a lógica global do estado por meio de uma instância de
 966 `app`, permitindo interação com outros módulos e componentes.

967 Atributos

968 A classe `Module` possui os seguintes atributos:

- 969 • **name**: Uma string que representa o nome do módulo. É utilizada para identificar o
 970 módulo de forma única.
- 971 • **app**: Uma instância de `App` que representa a aplicação principal. Esse atributo
 972 permite a interação com outros módulos e componentes, mas deve ser utilizado com
 973 cuidado para evitar acoplamento excessivo.
- 974 • **State**: Uma instância de `State` que armazena o estado do objeto observado. Esse
 975 estado é utilizado para observar mudanças notificadas pelas chaves `on_untrack` e
 976 `on_commit`.

[vii] https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/src/ui/components/shared/sidebar_item/widget.py

977 Operações

978 As operações da classe `Module` incluem:

- 979 • `__init__`: Inicializa o `Module` com uma instância de `app` e um `controller` associado
980 ao tipo de estado especificado. A instância de `app` representa a aplicação principal,
981 possibilitando a interação com outros módulos e componentes.
- 982 • `show`: Exibe o(s) `widget(s)` associado(s) ao `Controller`.
- 983 • `hide`: Oculta o(s) `widget(s)` associado(s) ao `Controller`.
- 984 • `reset`: Restaura os valores do `widget` e do estado para os valores padrão. Após a
985 execução, o estado é automaticamente rastreado.
- 986 • `load`: Atualiza os valores do `widget` e do estado com os dados fornecidos no parâmetro
987 `value` do tipo `State`. Assim como no método `reset`, o estado é rastreado após a
988 execução.
- 989 • `subscribe`: Adiciona um *handler* para notificações de mudanças no estado das
990 chaves `on_untrack` e `on_commit`. O *handler* é uma função de *callback* que recebe os
991 valores antigo e novo do estado. Para centralizar a lógica do domínio, os *handlers*
992 são declarados como métodos estáticos da classe derivada de `State`.

993 Com base no `Controller` apresentado na Seção 3.3.1, é apresentado a seguir a imple-
994 mentação do `Module` correspondente, Código 3.3.

```

995 @FAMetaCheckModule                                     11
996 class KMeansSolverModule(FAModule[KMeansSolverController, KMeansSolverState]): 12
997     """KMeans Solver module.""" 13
998 14
999     name: str = "KMeansSolver" 15
1000     app: "FeaturesAnalyzer" 16
1001     controller: KMeansSolverController 17
1002 18
1003     def __init__(self, app: "FeaturesAnalyzer", state: KMeansSolverState) -> None: 19
1004         super().__init__(app=app, controller=KMeansSolverController(state)) 20
1005 21
1006     def subscribe(self) -> None: 22
1007         """Load the module subscribers.""" 23
1008         # On track changes, update the module 24
1009         # show button to commit changes 25
1010         self.controller.widget.handle_status(state=self.state) 26
1011 27
1012         with self.state as state: 28
1013             state.on_change( 29
1014                 "on_commit", 30
1015                 lambda _: state.handle_001_on_params_update(self.app), 31
1016             ) 32
1017 33
1018             state.on_change("on_commit", lambda _: self.app.store.dump()) 34
1019 35
1020             state.on_change( 36
1021                 "on_untrack", 37

```

Código 3.3: Exemplo de implementação do `Module`.

Fonte: O Próprio Autor (2025)

1022

1023

1024 Na linha 11, observa-se novamente o decorador `@FAMetaCheckModule`, que verifica se
1025 os atributos necessários e seus respectivos tipos estão presentes na classe derivada. A
1026 declaração da classe na linha 12 define a `KMeansSolverModule`, que herda de `Module`

com os tipos `KMeansSolverController` e `KMeansSolverState`. Essa abordagem permite manter referências para ambos os tipos de estado e `controller` associados ao módulo.

Além disso, na linha 16, a tipagem de `app` é feita entre aspas devido à necessidade de definição explícita para verificação do código. A classe `FeaturesAnalyzer`, por ser uma das primeiras a ser instanciadas, é importada apenas para verificação de tipo, evitando dependências cíclicas.

Na linha 19, o `Module` é inicializado com uma instância de `app` e um `controller` correspondente ao tipo de estado associado. A inicialização da classe base é garantida pela chamada `super().__init__()`. Como a classe base já invoca o método `subscribe` na linha 22, não é necessário chamá-lo explicitamente.

Por fim, destaca-se o exemplo de conexão entre as atualizações de estado global pelas chaves `on_untrack` e `on_commit`, utilizando gerenciadores de contexto no método `subscribe`, localizado na linha 22. Por meio da cláusula `with`, o uso de gerenciadores de contexto assegura que as atualizações de estado sejam realizadas de forma segura e consistente, mitigando problemas de inconsistência causados por falhas nos `handlers`. Caso ocorra algum erro, o estado é restaurado utilizando o atributo `_backup`. A implementação desse gerenciador de contexto encontra-se nos métodos `__enter__` e `__exit__` da classe `State`.

No contexto mencionado, a chave `on_untrack` conecta-se ao `handler on_module_change` do widget, que atualiza a interface do usuário adicionando um botão para aplicar alterações por meio do método `state.commit()`. Esse botão, por sua vez, aciona o método `commit()` através de `handle_status()` do widget. A implementação detalhada desse método pode ser encontrada no arquivo [src/ui/components/shared/sidebar_item/widget.py](#) ?? mencionado anteriormente.

Já a chave `on_commit`, associada ao `handler handle_001_on_params_update`, aplica as alterações de parâmetros ao modelo após o acionamento do botão. Adicionalmente, essa mesma chave conecta-se ao método `dump` da classe `Store`, que consolida todos os estados da aplicação em um único contexto. Na Seção 3.4, serão apresentados mais detalhes sobre a classe `Store`. Por fim, na Figura 3.6 é apresentado o diagrama de arquitetura de componente, ilustrando a organização entre o `Module`, `Controller`, `State` e a `Widget`.

3.4 Store

A classe `Store` desempenha um papel fundamental no gerenciamento de estados globais da aplicação. Com o intuito de centralizar os contextos em uma única instância, a `Store` facilita a manipulação e a persistência de estados, fornecendo uma interface para salvá-los em arquivos e, posteriormente, carregá-los. A implementação dessa classe segue o padrão de projeto *Singleton*, garantindo que apenas uma instância esteja disponível em todo o ciclo de vida da aplicação.

A `Store` utiliza uma instância de `State` e encapsula o estado global da aplicação a partir dos estados individuais de cada componente. Uma vez armazenados na `Store`, os estados podem ser facilmente acessados e manipulados por outros módulos e componentes, simplificando a interação entre eles.

Para permitir o acesso nas demais camadas à `Store`, a instância é armazenada em um atributo da classe `App`, passada como parâmetro para os módulos e `handlers` de estado. Outra funcionalidade importante da `Store` é a capacidade de serializar todos os estados em um objeto e persistir em um arquivo quando necessário.

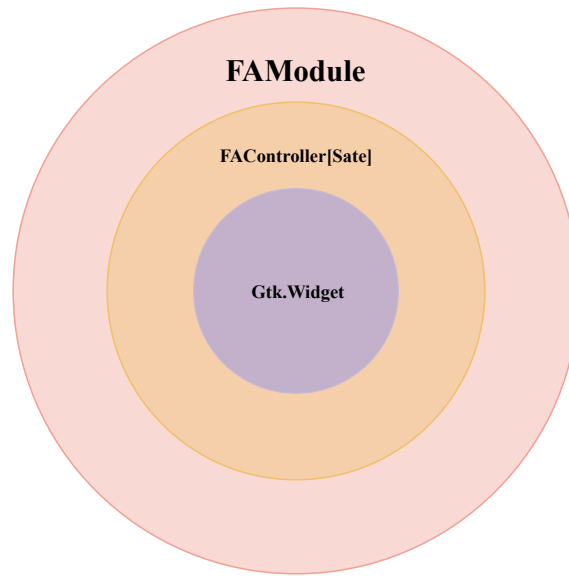


Figura 3.6: Diagrama de Arquitetura de Componente.
Fonte: O Próprio Autor (2025)

```

1072 class StateType(FASate):
1073     """Global application state."""
1074
1075     data: ClassVar[dict[str, pd.DataFrame]]
1076
1077     ImportSettings: ImportSettingsState
1078     KMeansSolver: KMeansSolverState
1079     MLPSolver: MLPSolverState
1080
1081
1082 class Store(metaclass=SingletonMeta):
1083     """State class for storing global state."""
1084
1085     __state = StateType(
1086         state={
1087             "data": {},
1088             "ImportSettings": ImportSettingsState(),
1089             "KMeansSolver": KMeansSolverState(),
1090             "MLPSolver": MLPSolverState(),
1091         },
1092     )
1093
1094     def __init__(self) -> None:
1095         self.load()
1096
1097     @property
1098     def state(self) -> StateType: ...
1099
1100     # Dump state to json.
1101     def dump(self) -> None: ...
1102
1103     # Load state from json
1104     def load(self) -> None: ...
1105

```

Código 3.4: Pseudo-Implementação da classe **Store**.

Fonte: O Próprio Autor (2025)

1106

1107

1108 A seguir, destacam-se algumas das principais responsabilidades da **Store**, Código 3.4:

1109

- Centralizar todos os estados globais em um único contexto.

- Persistir o estado em um banco de dados utilizando o método `dump`. O estado é serializado e armazenado em um formato estruturado, JSON, garantindo a integridade dos dados.
- Restaurar o estado global por meio do método `load`, que busca os dados previamente salvos no banco e os aplica ao contexto atual.
- Garantir a consistência e a integridade do estado global, com suporte para operações como rastrear e descartar mudanças nos estados associados.

Com a demanda por persistir os estados globais, e futuramente informações sobre os modelos de processamento de dados, surgiu a necessidade de integrar um banco de dados e um ORM (*Object-Relational Mapping*).

Essa integração não apenas simplifica o armazenamento e a recuperação de estados, mas também garante que os dados sejam organizados e acessíveis de maneira eficiente. Mais detalhes sobre o banco de dados utilizado serão apresentados na Seção 3.5.

A classe `Store` é um componente central no framework desenvolvido, desempenhando um papel crucial na gestão e persistência do estado global da aplicação. Sua integração com o banco de dados, o uso de *logs* (registro de eventos) e a adoção de boas práticas de desenvolvimento reforçam a robustez e a manutenibilidade do sistema.

3.5 Banco de Dados

Nesta seção, é apresentada a solução implementada para o armazenamento de dados de forma tipada utilizando a biblioteca `TinyDB`, em conjunto com `Pydantic`, para a definição de *schemas* de dados. O `TinyDB` foi escolhido como base para o armazenamento de dados devido à sua simplicidade, flexibilidade e suporte nativo ao formato JSON.

A implementação mantém a interface padrão do `TinyDB`, adicionando apenas uma camada de tipagem para parâmetros e retornos baseada em *schemas*, assegurando maior segurança e clareza no manuseio dos dados.

Cada banco é armazenado em um arquivo JSON distinto, cujo caminho pode ser configurado por meio de variáveis de ambiente. No entanto, um mesmo banco pode conter um conjunto de tabelas. Por exemplo, a tabela de configurações, ou estados da interface gráfica, `settings` é configurada conforme apresentado no Código 3.5.

```
settings_db = TypedTinyDB(path=os.getenv("SETTINGS_DB_PATH", "../settings.json")) 1
SettingsTable = settings_db.table("Settings", SettingsSchema) 2
# table = db.table(name="table-name", schema=Schema) 3
4
5
```

Código 3.5: Exemplo de configuração de um banco de dados e uma tabela de configurações.

Fonte: O Próprio Autor (2025)

1146

1147

Para mais informações sobre o funcionamento geral do `TinyDB`, recomenda-se consultar a documentação oficial da biblioteca `TinyDB` (2025).

1149

3.5.1 Definição de *Schemas* com Pydantic

A tipagem dos dados é realizada com a biblioteca `Pydantic` (Pydantic (2025)), que permite a validação e a conversão dos dados com base em *schemas* definidos como classes Python. *Schemas* são utilizados para definir a estrutura dos dados, incluindo os tipos dos campos, valores padrão e restrições de validação. Na Subseção 3.5.2 é apresentado a forma de integração entre `Pydantic` e `TinyDB`. No Código 3.6 é apresentado um exemplo de como criar um *schema* base e um *schema* específico para uma tabela de pré-processadores de dados.

```

1160 from pydantic import BaseModel as PydanticBaseModel
1161
1162
1163 class BaseSchema(PydanticBaseModel):
1164     """Base class for FAORM models."""
1165     uid: str | None = None
1166
1167
1168 class FAPreProcessorSchema(BaseSchema):
1169     """A preprocessor to prepare data for a model."""
1170     name: str
1171     params: Any
1172     data_hash: str | None = None
1173

```

Código 3.6: Exemplo de definição de *schemas* com Pydantic.

Fonte: O Próprio Autor (2025)

3.5.2 Tabelas Tipadas

A funcionalidade de tabelas tipadas é implementada por meio de uma classe personalizada, `TypedTinyDB`, que estende o comportamento do `TinyDB` para validar e gerenciar *schemas*. A implementação dessa classe é realizada através da extensão e **overriding** (sobrescrita) de métodos da classe `TinyDB`, garantindo a compatibilidade com a interface padrão da biblioteca. As principais alterações para que a tabela atenda à tipagem dos *schemas* é descrita na Lista 3.5.2.

- **generate_hash**: Método para gerar um *hash* único para cada registro, baseado nos dados fornecidos. Esse *hash* é utilizado para identificar de forma única cada registro na tabela.

A técnica de *hashing* utilizada é utilizada para garantir a unicidade dos registros e facilitar a busca e a atualização dos dados. A abordagem consiste em concatenar os valores dos campos do registro e aplicar uma função que cria um valor único e determinístico, como o *hash* SHA-256.

- **find_by_uid**: Método para buscar um registro na tabela com base no *uid* fornecido. Caso o registro não seja encontrado, `None` é retornado.
- **insert**: Método para inserir um novo registro na tabela.
- **update**: Método para atualizar um registro existente na tabela.
- **all**: Método para retornar todos os registros da tabela.

1193 • **search**: Método para buscar registros na tabela com base em um critério de pesquisa
1194 fornecido.

1195 Para mais detalhes sobre a implementação da classe `TypedTinyDB`, recomenda-se con-
1196 sultar o código-fonte do projeto, no [arquivo^{\[viii\]}](#).

1197 Uma vez definidos os **schemas**, os bancos e tabelas são instanciados no [arquivo^{\[ix\]}](#).

1198 3.5.3 Configurações e Arquitetura

1199 A configuração de cada banco de dados é realizada por meio de caminhos para os
1200 arquivos; estes podem ser configurados por variáveis de ambiente, permitindo flexibilidade
1201 no gerenciamento dos dados em diferentes ambientes.

1202 A decisão de utilizar o `TinyDB` como base para o armazenamento de dados foi motivada,
1203 além da simplicidade, devido ao uso de arquivos `JSON`, que facilitam a integração com
1204 outras ferramentas, a portabilidade dos dados, a edição e a visualização dos registros. Um
1205 dos principais usos do banco de dados é apresentado na Seção 3.6.

1206 3.6 Gerenciamento de Modelos

1207 O gerenciamento de modelos é uma prática essencial em projetos de análise de da-
1208 dos, aprendizado de máquina e sistemas que demandam iterações contínuas com dados.
1209 Essa abordagem organiza, armazena, carrega e reutiliza modelos eficientemente, garan-
1210 tindo consistência nos resultados e alta manutenibilidade. Em cenários que envolvem um
1211 volume de dados considerável, o gerenciamento de modelos é fundamental para lidar com
1212 a complexidade dos algoritmos e o tempo de execução das análises.

1213 A capacidade de salvar e carregar modelos evita treinamentos repetitivos, assegura
1214 a consistência dos experimentos e facilita a comparação entre diferentes configurações.
1215 Além disso, a implementação desenvolvida oferece meios de compartilhar modelos entre
1216 colaboradores, promovendo o desenvolvimento colaborativo, auditoria e replicação de ex-
1217 perimentos.

1218 Na arquitetura desenvolvida, o gerenciamento de modelos está diretamente relacionado
1219 à camada de estados, instanciado pelos **handlers** responsáveis por integrar os modelos com
1220 o restante da aplicação. Nesta seção, está detalhado a estrutura e os benefícios da classe
1221 `FAModel`, utilizando como exemplo o modelo `KMeans`.

1222 3.6.1 Estrutura do Gerenciador de Modelos

1223 A classe base `FAModel` foi projetada como uma solução genérica e extensível, possibili-
1224 tando a integração com diversos algoritmos de processamento de dados e aprendizado de
1225 máquina. Sua implementação utiliza `TypeVar` para parametrização genérica e `Pydantic`
1226 para validação e consistência dos dados. O exemplo do modelo `KMeans` 3.7, será usado
1227 para ilustrar sua aplicação.

[viii] <https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/src/lib/orm/table.py>

[ix] <https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/src/lib/orm/db.py>

Inicialização e Configuração

Antes de utilizar o gerenciador, é necessário configurar o ambiente para armazenamento:

- **Diretório de Modelos:** Configurado pela variável `BIN_PATH`, com valor padrão `./bin`. Se inexistente, o diretório será criado automaticamente.
- **Banco de Dados:** Configurado pela variável `MODELS_DB_PATH`, com valor padrão `./db.json`. Esse arquivo armazena metadados como parâmetros e identificadores dos modelos.

Após a configuração, um gerenciador pode ser definido para qualquer algoritmo. A implementação da classe `KMeans`, derivada de `FAModel`, exemplifica o gerenciamento do modelo `KMeans` do `scikit-learn`. A seguir, é apresentado um trecho do código relevante:

```

1239 from typing import Literal
1240
1241 import numpy as np
1242 from sklearn.base import check_is_fitted
1243 from sklearn.cluster import KMeans as SklearnKMeans
1244 from sklearn.exceptions import NotFittedError
1245
1246 from lib.model.manager import FAModel
1247 from lib.orm.schemas import PydanticBaseModel
1248
1249
1250 class KMeansParams(PydanticBaseModel):
1251     """Parameters for the KMeans model."""
1252
1253     n_clusters: int
1254     max_iter: int = 300
1255     n_init: int = 10
1256     init: Literal["k-means++", "random"] = "k-means++"
1257     random_state: int = 0
1258
1259
1260 class KMeans(FAModel[SklearnKMeans, KMeansParams]):
1261     """KMeans model manager."""
1262
1263     schema = KMeansParams
1264
1265     def __init__(self, x: np.ndarray, params: KMeansParams | None) -> None:
1266         """Initialize the model."""
1267         self.store(x, params)
1268
1269         if not self.loaded():
1270             self.method = SklearnKMeans(
1271                 n_clusters=self.params.n_clusters,
1272                 max_iter=self.params.max_iter,
1273                 init=self.params.init,
1274                 random_state=self.params.random_state,
1275             )
1276
1277     def fit(self, x: np.ndarray) -> None:
1278         """Fit the model."""
1279         try:
1280             check_is_fitted(self.method)
1281         except NotFittedError:
1282             self.method.fit(x)
1283             self.save()
1284
1285     def predict(self, x: np.ndarray) -> np.ndarray:
1286         """Predict the clusters."""
1287         return self.method.predict(x)

```

Código 3.7: Implementação do Modelo KMeans

Fonte: O Próprio Autor (2025)

Os principais elementos dessa implementação são detalhados abaixo:

- 1291 • **Definição de Parâmetros:** Nas linhas 12-20, o esquema de parâmetros é definido
1292 utilizando `PydanticBaseModel`. Essa abordagem garante que os parâmetros sejam
1293 validados e consistentes com o algoritmo de análise definido.
- 1294 • **Classe do Modelo:** Nas linhas 22, a classe `KMeans` é definida como uma especia-
1295 lização de `FAModel`, parametrizada pelos tipos `SklearnKMeans` e `KMeansParams`, que
1296 representam, respectivamente, o modelo do `scikit-learn` e os parâmetros.
- 1297 • **Inicialização do Modelo:** O método `__init__` (linhas 27-37) é responsável por
1298 verificar a existência do modelo no armazenamento. Caso não exista, ele registra os
1299 parâmetros no banco de dados, inicializa o modelo e persiste os dados.
- 1300 • **Treinamento:** O método `fit` (linhas 39-45) realiza o ajuste do modelo aos dados
1301 fornecidos. Após o treinamento, o modelo é salvo no diretório especificado por
1302 `BIN_PATH`, utilizando um nome padronizado: `<model_name>.<datetime>.<uid>.pkl`.
1303 Essa nomenclatura inclui o nome do modelo, a data/hora e um identificador único.
- 1304 • **Predição:** O método `predict` (linhas 47-49) utiliza o modelo treinado para re-
1305 alizar previsões. Antes da predição, o método verifica se o modelo foi ajustado
1306 corretamente.

1307 3.6.2 Benefícios do Gerenciamento de Modelos

1308 A abordagem para o gerenciamento de modelos, utilizando a classe `FAModel`, oferece
1309 diversos benefícios para projetos de análise de dados:

- 1310 • **Reutilização:** Modelos podem ser salvos e carregados para evitar treinamentos
1311 desnecessários. Além disso, os modelos podem ser comparados, para, por exemplo,
1312 atender a necessidades de ajuste de parâmetros.
- 1313 • **Consistência:** A validação de parâmetros e identificadores únicos assegura integri-
1314 dade no armazenamento.
- 1315 • **Flexibilidade:** A parametrização genérica permite a integração com diferentes al-
1316 goritmos. Logo, é possível integrar facilmente qualquer método de análise, uma vez
1317 conhecido e definindo os parâmetros.
- 1318 • **Escalabilidade:** A estrutura modular facilita a adaptação a novas funcionalidades
1319 e cenários. Na implementação descrita, por exemplo, é possível também utilizar
1320 pré-processadores antes da etapa de treinamento, e da mesma forma, registrar o uso
1321 com um identificador único dos parâmetros e resultado do pré-processamento. Outra
1322 possibilidade, seria integrar um módulo de controle de métricas de avaliação.
- 1323 • **Colaboração:** A possibilidade de compartilhar modelos promove a replicação de
1324 experimentos e auditorias. Um exemplo prático seria um membro de pesquisa con-
1325 seguir compartilhar um modelo complexo, após o treinamento, com toda a equipe.

1326 A implementação do `KMeans` demonstra como a arquitetura genérica da `FAModel` pode
1327 ser aplicada a algoritmos específicos, garantindo uma integração eficiente e confiável.

3.7 Interface Gráfica do *Boilerplate*

Além de todos os componentes e abstrações apresentados, a interface gráfica é um dos principais elementos do *boilerplate*. A interface foi desenvolvida utilizando a biblioteca `PyGObject`, que fornece uma API para a criação de interfaces gráficas em Python. A seguir, são apresentadas as principais características e funcionalidades da interface gráfica do *boilerplate* ilustradas na Figura 3.7.

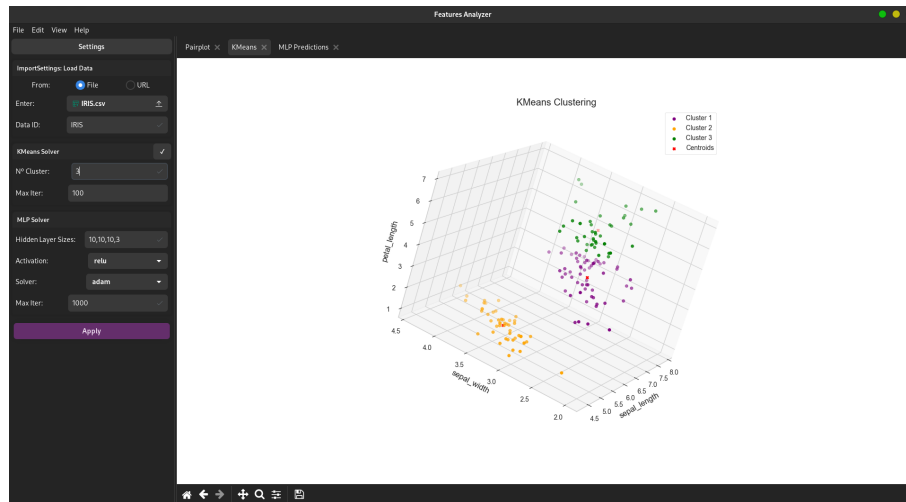


Figura 3.7: Interface Gráfica do *Boilerplate*
Fonte: O Próprio Autor (2025)

- **Barra de Ferramentas:** Localizada na parte superior da janela, a barra de ferramentas contém botões para ações comuns, como abrir, salvar e executar, além de menus para configurações e ajuda. Esse componente deve ser ajustado para atender às necessidades específicas de cada aplicação.
- **Barra Lateral (*sidebar*):** À esquerda da janela, a barra lateral exibe os componentes disponíveis para interação, como configurações de importação e parâmetros de modelos. A responsabilidade pela importação dos dados para a memória é gerenciada pela biblioteca `Pandas`.
- **Área de Trabalho:** O espaço central da janela é dedicado à exibição de gráficos. O componente consiste em um *widget* de `Gtk.Notebook` que permite alternar entre diferentes visualizações.

A interface gráfica é completamente customizável e extensível, permitindo a adição de novos componentes, personalização de estilos e integração com outras bibliotecas gráficas.

A arquitetura do *boilerplate* facilita a criação de interfaces intuitivas e eficientes, oferecendo como exemplo a implementação do `KMeans` e `MLP` (*Multi-Layer Perceptron* ou *Perceptron de Múltiplas Camadas*) para tratar o famoso conjunto de dados flor Iris.

3.8 Utilitários

Nessa seção é apresentado os utilitários desenvolvidos para facilitar a implementação e o uso do sistema. Eles são responsáveis por simplificar a execução de tarefas comuns, como a auditoria do sistema, documentação, análise e formatação, tradução e internacionalização, etc.

3.8.1 Logger

Devido ao grande número de operações e a complexidade da aplicação, surgiu a necessidade de um *logger*, uma ferramenta de uso genérico para auditoria. Ela é responsável por registrar informações sobre o funcionamento do sistema, tanto no terminal de execução atual quanto em um arquivo de log dedicado. Essa funcionalidade facilita a auditoria, depuração e monitoramento da aplicação.

Os logs gerados pelo **logger** são categorizados em diferentes níveis, cada um com uma cor associada para facilitar a identificação no terminal:

- **DEBUG**: Informações detalhadas para diagnóstico. Representado na cor azul.
- **INFO**: Informações gerais sobre o funcionamento da aplicação. Representado na cor cinza.
- **SUCCESS**: Mensagens que indicam operações bem-sucedidas. Representado na cor verde.
- **WARNING**: Alerta sobre situações que podem exigir atenção, mas que não são erros. Representado na cor amarela.
- **ERROR**: Mensagens que indicam falhas que impedem a execução de uma funcionalidade. Representado na cor vermelha.
- **CRITICAL**: Erros graves que requerem atenção imediata. Representado na cor magenta.

Os logs são utilizados em vários pontos da aplicação, como na inicialização do sistema, na restauração dos estados, treinamento de modelos e persistência de dados. No trecho a seguir 3.8 é apresentado alguns exemplos simples do uso do **logger**.

```
logger.info(f"{PREFIX} State dumped.")
logger.info(f"{PREFIX} State loaded.")
logger.success(f"{self._PREFIX} - Model loaded from {self.file_name}.")
logger.info(f"{self._PREFIX} - Model binary not found.")
logger.info(f"{self._PREFIX} - Model entry not found.")
logger.error(f"{self._PREFIX} - {e}")
logger.warning(f"{self.LOG_PREFIX} Document with UID {uid} not found")
```

Código 3.8: Exemplos de uso do **logger**

Fonte: O Próprio Autor (2025)

3.8.2 Documentação

A documentação desempenha um papel essencial na manutenção e na extensibilidade de sistemas complexos, como o desenvolvido neste trabalho. Para garantir que os desenvolvedores possam facilmente personalizar e compreender o funcionamento do sistema, foi adotada uma estratégia baseada em **Markdown** com o uso das ferramentas [MKDocs](https://www.mkdocs.org/)^[x] e **mkdocs-material**. Essa abordagem simplifica a criação de uma documentação rica e interativa, que pode ser disponibilizada em formato de site.

A implementação do sistema inclui um gerador automatizado que converte arquivos **.md** em páginas HTML, CSS e JavaScript estilizadas com o tema **mkdocs-material**. O resultado é um site responsivo e bem estruturado, armazenado em uma *branch* **gh-pages** do repositório no GitHub.

Essa *branch* serve como base para a publicação automática do site, tornando a documentação acessível para toda a equipe. As configurações do **MKDocs** são definidas no arquivo **mkdocs.yml**, onde é possível personalizar o tema, a navegação, *plugins* e outros aspectos.

Workflow e CI/CD

Para automatizar o processo de geração e *deploy* da documentação, foi configurado um **workflow** utilizando os recursos de integração contínua (CI) e entrega contínua (CD) disponíveis no GitHub. Um **workflow** consiste em um conjunto de etapas definidas em um arquivo **YAML**, armazenado no diretório **.github/workflows** do repositório. Ele descreve os gatilhos (como *commit's*, *pull requests* ou *tags*) e as ações que devem ser executadas, como a instalação de dependências, a execução de testes e a implantação.

O conceito de **CI/CD** (Integração Contínua e Entrega Contínua) refere-se a práticas que permitem o desenvolvimento, o teste e a implantação de software de forma automática e iterativa. Na integração contínua, alterações no código são frequentemente integradas ao repositório principal, garantindo que ele esteja sempre atualizado e testado. Na entrega contínua, essas alterações são automaticamente implantadas em ambientes de produção ou *staging*, facilitando a liberação de novas funcionalidades.

No caso do gerador de documentação, o workflow configurado no GitHub executa as seguintes etapas:

1. **Gatilho:** O processo é iniciado após um *push* na branch principal do repositório.
2. **Configuração do Ambiente:** As dependências necessárias para o **MKDocs** e o tema **mkdocs-material** são instaladas.
3. **Geração do Site:** O conteúdo em Markdown é processado e convertido em páginas HTML estilizadas.
4. **Deploy Automatizado:** O site gerado é enviado para a branch **gh-pages**, que serve como base para a publicação no **GitHub Pages**.

O **GitHub Pages** permite hospedar sites estáticos diretamente de um repositório no GitHub, sendo uma solução ideal para publicar documentações. Para ativá-lo, acesse as

^[x]<https://www.mkdocs.org/>

configurações do repositório, selecione a aba *Pages*, configure a *branch* e o diretório onde os arquivos estão armazenados, e salve as alterações.

Após a configuração, o site será publicado em um endereço gerado automaticamente, como `https://<username>.github.io/<repository>`, tornando a documentação acessível online de forma prática. Mais detalhes podem ser encontrados na [documentação oficial](#)^[xi].

Essa abordagem automatizada reduz o esforço manual, garante consistência no processo de *deploy* e promove uma documentação atualizada e acessível, essencial para o sucesso de projetos colaborativos. Embora o *workflow* simplifique o processo, ele pode ser feito manualmente sem grandes dificuldades, seguindo a [documentação](#)^[xii].

Com este utilitário, os desenvolvedores podem escrever a documentação dos próprios experimentos no projeto e hospedá-los na *web* de forma simples.

3.8.3 Análise e Formatação de Código

A *Ruff* é uma ferramenta moderna de *linting* (análise de código) e formatação de código desenvolvida para Python. Ele é projetado para ser extremamente rápido e oferecer suporte a uma ampla gama de regras e convenções de codificação, garantindo que o código permaneça legível, consistente e aderente às melhores práticas [Ruff \(2025\)](#).

Um *linter* é uma ferramenta que analisa o código-fonte em busca de erros, padrões não recomendados e violações de estilo. Já um **formatter** formata automaticamente o código para que ele siga padrões predefinidos, como indentação, espaçamento e quebras de linha, facilitando a padronização do projeto.

O *Ruff* destaca-se por oferecer suporte a várias regras e convenções de ferramentas amplamente utilizadas, como `flake8`, `isort`, `pylint`, entre outras. Isso permite integrar múltiplas funcionalidades em uma única ferramenta, simplificando o pipeline de desenvolvimento.

As configurações do *Ruff* são definidas no arquivo `ruff.toml`, onde podem ser especificadas regras habilitadas, exclusões e personalizações. Esse arquivo centralizado torna o gerenciamento de padrões de estilo mais acessível e compartilhável entre diferentes desenvolvedores de um projeto. Para mais informações, a documentação oficial do *Ruff*^[xiii].

3.8.4 Tradução e Internacionalização (i18n)

A internacionalização, comumente abreviada como *i18n*^[xiv], refere-se ao processo de adaptação de um sistema de *software* para suportar múltiplos idiomas e culturas sem a necessidade de reengenharia. Esse recurso é essencial para aumentar a acessibilidade do sistema, possibilitando que usuários de diferentes regiões utilizem a aplicação em seus idiomas nativos.

Na implementação, a funcionalidade de *i18n* é gerenciada por arquivos de tradução, que mapeiam as chaves do texto para suas correspondentes localizações. Esses arquivos, frequentemente escritos no formato JSON ou PO, permitem que o conteúdo da interface do usuário seja facilmente traduzido e adaptado para diferentes contextos culturais.

^[xi] <https://docs.github.com/en/pages>

^[xii] <https://docs.github.com/en/actions>

^[xiii] <https://docs.astral.sh/ruff/>

^[xiv] <https://pypi.org/project/python-i18n/>

Para o gerenciamento e aplicação de traduções, foi utilizada a biblioteca `python-i18n`. Essa biblioteca fornece uma interface simples e eficiente para carregar arquivos de tradução, selecionar idiomas e interpolar valores dinâmicos em strings traduzidas.

Além disso, ela suporta recursos avançados, como *fallback* de idiomas (uma linguagem padrão, quando não definida), facilitando a construção de sistemas robustos e internacionalizados.

A configuração da *i18n* utiliza ferramentas específicas que detectam o idioma do sistema ou permitem que o usuário selecione sua preferência. Além disso, o sistema foi projetado para facilitar a inclusão de novos idiomas por meio da simples adição de arquivos de tradução e ajustes mínimos no código.

Essa abordagem promove uma maior inclusão e acessibilidade do sistema, permitindo que usuários de diferentes origens e idiomas possam interagir com a aplicação de forma eficaz e intuitiva.

Sendo assim, instâncias desse *boilerplate* podem ser customizadas para incluir suporte a diferentes idiomas, conforme a necessidade do projeto, útil para aplicações desenvolvidas por equipes multilinguísticas. Na Seção 3.9 é apresentado as considerações finais deste capítulo.

3.9 Considerações Finais

Neste capítulo, é apresentada as principais abstrações e componentes do framework desenvolvido, incluindo os conceitos de *State/Observer*, *Widget*, *Controller*, *Module*, *Store*, banco de dados e gerenciamento de modelos.

A implementação seguiu princípios de design orientado a objetos, como encapsulamento, polimorfismo e herança, para garantir modularidade, reutilização e manutenibilidade do código. Abaixo, é descrito como cada componente contribui para a arquitetura do sistema e os métodos de implementação e integração adotados.

1. **State/Observer:** Facilita a comunicação entre componentes. O **State** representa o estado de um componente, enquanto o **Observer** notifica mudanças de estado aos demais. Esse componente é essencial para a implementação de todas as funcionalidades.
2. **Store:** Gerencia e persiste os estados globais da aplicação. Centraliza os estados em um único contexto, simplificando sua manipulação e persistência, além de oferecer suporte ao salvamento em arquivos. Segue o padrão de projeto *Singleton*.
3. **Widget:** Interage com o usuário e exibe informações. Cada **Widget** está associado a um **State** e é notificado sobre alterações. Parâmetros ajustáveis são representados por *widgets* filhos, como o `Gtk.Entry`, usado para entradas de texto ou valores numéricos.
4. **Controller:** Conecta a interface gráfica ao **State**, gerenciando exibição, ocultação e conexões entre sinais dos *widgets* e *handlers* para validação e formatação dos valores.
5. **Module:** Garante a consistência do estado dentro de cada domínio, propagando alterações conforme necessário. Encapsula a lógica global do estado, interage com outros módulos e componentes e gerencia notificações de mudanças, como `on_untrack` e `on_commit`.

1505 6. **Model:** Gerencia e persiste os modelos de análise de dados. Implementado com
1506 base em *schemas* de validação, assegura a integridade dos modelos. Utiliza a classe
1507 **FAModel**, que fornece métodos genéricos como salvar, carregar e identificar modelos.
1508 Um exemplo é o uso do *KMeans*.

1509 7. **Handlers de Estado:** Atualizam a interface gráfica, persistem estados e gerenciam
1510 modelos de análise por meio de um ou mais *Models*.

1511 Esses métodos, estáticos na classe derivada de **State**, centralizam a lógica de domínio.
1512 Por exemplo, no estado do modelo **KMeans**, o *handler* gerencia as mudanças associ-
1513 adas a esse algoritmo. O [arquivo^{\[xv\]}](#) fornece um exemplo da utilização dos *handlers*
1514 com os modelos.

1515 O capítulo destacou as contribuições de cada componente para a arquitetura do sis-
1516 tema, demonstrando como a integração entre eles proporciona uma experiência de usuário
1517 consistente e eficiente. Após toda a apresentação, a arquitetura ilustrada na Figura 3.2
1518 complementa a visão geral sobre o sistema.

^[xv] https://github.com/ZauJulio/FeaturesAnalyzer/blob/test/src/context/states/kmeans_solver.py

Capítulo 4

Conclusões

Este trabalho apresentou o desenvolvimento de um *boilerplate* para a criação de aplicações e ferramentas interativas destinadas à análise e visualização de dados. A solução proposta combina ferramentas amplamente utilizadas, como `GTK`, `matplotlib`, `pandas` e `scikit-learn`, em uma arquitetura modular e extensível. Neste capítulo, é apresentado os resultados obtidos e listado possíveis trabalhos futuros que podem expandir ou aprimorar o *boilerplate* inicial.

4.1 Resultados

O principal resultado deste trabalho foi o desenvolvimento de uma estrutura base que simplifica a criação de ferramentas de análise e visualização de dados. Entre as principais funcionalidades, destacam-se:

- Interface modular e extensível: A utilização do `GTK` para a criação da interface gráfica permitiu a construção de dashboards interativos, organizados em módulos independentes.
- Visualização eficiente de dados: A integração com o `matplotlib` e `seaborn` garantiu uma visualização clara e customizável, possibilitando análises mais intuitivas para os usuários.
- Gerenciamento genérico de modelos: O gerenciador de modelo baseado no `FAModel` demonstrou ser eficaz para lidar com algoritmos de processamento de dados e aprendizado de máquina, como o `KMeans`, permitindo armazenamento, carregamento e reutilização de modelos de maneira consistente e eficiente.
- Escalabilidade e reutilização: A arquitetura orientada a objetos proporcionou modularidade e reutilização de componentes, facilitando a extensão do *boilerplate* para novos algoritmos e funcionalidades.
- Documentação: A integração com `mkdocs` permitiu a criação de uma documentação clara e acessível, facilitando a documentação de experimentos, resultados e funcionalidades do sistema quando estendido.
- *linter* e *formatter*: A integração com `ruff` entrega uma ferramenta de formatação e verificação de código, garantindo a consistência e a qualidade do código produzido.

- Compartilhamento de arquivos JSON e modelos binários: A capacidade de exportar e importar dados e modelos em formatos comuns, como JSON e `pickle`, facilita a colaboração e a reutilização de resultados.

Os exemplos fornecidos demonstraram que o *boilerplate* é capaz de integrar diferentes algoritmos de processamento de dados e aprendizado de máquina, de forma simples e oferecer suporte a fluxos completos de análise de dados, desde a manipulação de entradas até a geração de resultados visuais.

4.2 Trabalhos Futuros

Apesar dos avanços alcançados, este trabalho abre espaço para diversas possibilidades de melhorias e expansões:

- Integração com outros frameworks: Explorar a integração com outras bibliotecas de visualização e aprendizado de máquina, como `Plotly` para gráficos interativos ou gerenciadores específicos para `TensorFlow` para modelos mais complexos.
- Processamento distribuído: Implementar suporte para processamento distribuído, permitindo a execução de análises em paralelo em múltiplos núcleos ou máquinas.
- Otimização de desempenho: Implementar melhorias para reduzir o consumo de memória ao lidar com grandes volumes de dados, como a utilização de *generators* e *streams*.
- Documentação expandida: Ampliar a documentação para incluir tutoriais, exemplos e guias de uso, facilitando a adoção e a extensão do sistema a partir do próprio repositório através do `mkdocs`. Essa estratégia também fornecerá exemplos de como documentar as próprias aplicações desenvolvidas com o *boilerplate*.
- Traduzir e internacionalizar: Adicionar suporte a outros idiomas e localizações, como espanhol, japonês e português, tornando o sistema acessível a um público mais amplo.
- Processamento em tempo real: Adicionar suporte para processamento em tempo real, permitindo a análise de dados em fluxo contínuo, como em aplicações de IoT ou monitoramento de sistemas.
- Suporte a múltiplas fontes de dados facilitado: Embora a biblioteca `pandas` já forneça conectores, a proposta de melhoria é expandir o sistema para suportar a integração com diferentes fontes de dados, como bancos de dados SQL, NoSQL ou APIs de serviços web, fornecendo os widgets necessários para interação com essas fontes.
- Interface web: Expandir a aplicação para suportar sistemas de visualização baseados na web, ampliando o alcance e a usabilidade do sistema.
- Automação de análises: Incorporar ferramentas para automação de análises preditivas e geração de relatórios, tornando o sistema ainda mais útil em contextos empresariais.
- Validação com usuários: Realizar testes de usabilidade com potenciais usuários para identificar melhorias na experiência e na interface do sistema.

- 1587 • Biblioteca de modelos: Desenvolver uma biblioteca ampliada de algoritmos prontos
1588 para uso, abrangendo métodos de regressão, classificação e análise de séries tempo-
1589 rais.

1590 Essas direções apontam para a evolução do *boilerplate* como uma solução completa e
1591 flexível para análise de dados, tanto em contextos acadêmicos quanto industriais.

Referências Bibliográficas

GTK (2024), ‘Gtk: The gimp toolkit’. Acessado em 09 janeiro 2025.

URL: <https://www.gtk.org/>

(Citado na página 9)

Guimarães, Eliane Marina Palhares e Yolanda Dora Martinez Évora (2004), ‘Sistema de informação: instrumento para tomada de decisão no exercício da gerência’, *Ciência da informação* **33**.

(Citado na página 6)

Han, Jiawei, Micheline Kamber e Jian Pei (2012), *Data Mining: Concepts and Techniques*, 3rd^a edição, Morgan Kaufmann, Waltham, MA, USA. Acessado em 14 janeiro 2025.

URL: <https://www.sciencedirect.com/book/9780123814791/data-mining-concepts-and-techniques>

(Citado na página 1)

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke e Travis E. Oliphant (2020), ‘Array programming with NumPy’, *Nature* **585**(7825), 357–362.

URL: <https://doi.org/10.1038/s41586-020-2649-2>

(Citado na página 8)

Ike-Nwosu, Obi (2017), ‘Read inside the python virtual machine’.

URL: <https://leanpub.com/insidethepythonvirtualmachine/read>

(Citado na página 8)

Introspection, GObject (2024), ‘Gobject introspection is a middleware layer between c libraries (using gobject) and language bindings.’. Acessado em 09 janeiro 2025.

URL: <https://gi.readthedocs.io/en/latest/>

(Citado na página 10)

Lämmel, Ralf e Simon Peyton Jones (2003), *Scrap your boilerplate*, ACM SIGPLAN, New Orleans, Louisiana, USA. Acessado em 14 janeiro 2025.

URL: <https://dl.acm.org/doi/10.1145/640136.604179>

(Citado na página 5)

- 1625 Monard, Maria Carolina e José Augusto Baranauskas (2003), ‘Conceitos sobre aprendizado
1626 de máquina’, *Sistemas inteligentes-Fundamentos e aplicações* **1**(1), 32.
1627 (Citado na página 6)
- 1628 Mypy (2024), ‘Mypy: Optional static typing for python’. Acessado em 09 janeiro 2025.
1629 **URL:** <https://mypy-lang.org/>
1630 (Citado na página 8)
- 1631 pandas development team, The (2020), ‘pandas-dev/pandas: Pandas’.
1632 **URL:** <https://doi.org/10.5281/zenodo.3509134>
1633 (Citado na página 9)
- 1634 Pedregosa, F., G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel,
1635 P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M.
1636 Brucher, M. Perrot e E. Duchesnay (2011), ‘Scikit-learn: Machine learning in Python’,
1637 *Journal of Machine Learning Research* **12**, 2825–2830.
1638 (Citado na página 9)
- 1639 Pydantic (2025), ‘Pydantic documentation’. Acessado em 14 janeiro 2025.
1640 **URL:** <https://docs.pydantic.dev/latest/>
1641 (Citado na página 33)
- 1642 PyGObject (2024), ‘Pygobject: Python bindings for gobject-based libraries’. Acessado
1643 em 09 janeiro 2025.
1644 **URL:** <https://pygobject.gnome.org/>
1645 (Citado na página 10)
- 1646 Ruff (2025), ‘Ruff’. Acessado em 16 janeiro 2025.
1647 **URL:** <https://docs.astral.sh/ruff/>
1648 (Citado na página 40)
- 1649 Tarapanoff, Kira (1995), ‘Técnicas para tomada de decisão nos sistemas de informação’.
1650 (Citado na página 1)
- 1651 TinyDB (2025), ‘Tinydb documentation - usage’. Acessado em 16 janeiro 2025.
1652 **URL:** <https://tinydb.readthedocs.io/en/latest/usage.html>
1653 (Citado na página 32)
- 1654 Van Rossum, Guido e Fred L Drake Jr (1995a), *Python reference manual*, Centrum voor
1655 Wiskunde en Informatica Amsterdam.
1656 (Citado na página 8)
- 1657 Van Rossum, Guido e Fred L Drake Jr (1995b), *Python tutorial*, Centrum voor Wiskunde
1658 en Informatica Amsterdam, The Netherlands.
1659 (Citado na página 8)
- 1660 Waskom, Michael (2024), ‘Seaborn: Statistical data visualization’, USA: Trelgol Pu-
1661 blishing. Acessado em 12 janeiro 2025.
1662 **URL:** <http://seaborn.pydata.org/>
1663 (Citado na página 9)